ĐỀ CƯƠNG CHI TIẾT ĐỒ ÁN MÔN BIG DATA

Khoa Công nghệ Thông tin — Trường Đại học Tân Tạo

MỤC LỤC

PHẦN A. QUY ĐỊNH CHUNG 4
A.1. Mục tiêu môn học được đánh giá qua đồ án 4
A.2. Tổ chức nhóm và thời lượng 4
A.3. Môi trường kỹ thuật bắt buộc 4
A.4. Cấu trúc thư mục HDFS chuẩn 5
A.5. Cấu trúc repository chuẩn 5
A.6. Yêu cầu bắt buộc chung (thiếu bất kỳ mục nào → không đạt) 5
A.7. Mốc tiến độ 5
A.8. Thang điểm chi tiết 6
A.9. Chống sao chép 6
PHẦN B. ĐỀ CƯƠNG CHI TIẾT TỪNG ĐỀ TÀI 7
ĐỀ TÀI 1 — HỆ THỐNG GIÁM SÁT VÀ DỰ BÁO PHỤ TẢI ĐIỆN 8
1.1. Bối cảnh nghiệp vụ 8
1.2. Nguồn dữ liệu 8
1.3. Kiến trúc hệ thống 8
1.4. Đặc tả bảng dữ liệu 9
1.5. Đặc tả các job Spark 10
1.6. Đặc tả mô hình học máy 11
1.7. Đặc tả Delta Lake 12
1.8. Đặc tả ứng dụng web 12
1.9. Đặc tả Airflow 13
1.10. Rủi ro và phương án dự phòng 13
1.11. Danh mục nghiệm thu 14
ĐỀ TÀI 2 — PHÁT HIỆN GIAN LẬN GIAO DỊCH THẺ THEO THỜI GIAN THỰC 15
2.1. Bối cảnh nghiệp vụ 15
2.2. Nguồn dữ liệu 15
2.3. Kiến trúc hệ thống 15
2.4. Đặc tả bảng dữ liệu 15
2.5. Đặc tả các job Spark 17

2.6. Đặc tả mô hình học máy 17
2.7. Đặc tả Delta Lake 18
2.8. Đặc tả ứng dụng web 18
2.9. Đặc tả Airflow 19
2.10. Rủi ro và phương án dự phòng 19
2.11. Danh mục nghiệm thu 20
ĐỀ TÀI 3 — HỆ KHUYẾN NGHỊ SẢN PHẨM CHO SÀN THƯƠNG MẠI ĐIỆN TỬ 21
3.1. Bối cảnh nghiệp vụ 21
3.2. Nguồn dữ liệu 21
3.3. Kiến trúc hệ thống 21
3.4. Đặc tả bảng dữ liệu 21
3.5. Đặc tả các job Spark 23
3.6. Đặc tả mô hình học máy 24
3.7. Đặc tả Delta Lake 24
3.8. Đặc tả ứng dụng web 25
3.9. Đặc tả Airflow 25
3.10. Rủi ro và phương án dự phòng 26
3.11. Danh mục nghiệm thu 26
ĐỀ TÀI 4 — PHÂN TÍCH VÀ DỰ BÁO NHU CẦU DI CHUYỂN ĐÔ THỊ 27
4.1. Bối cảnh nghiệp vụ 27
4.2. Nguồn dữ liệu 27
4.3. Kiến trúc hệ thống 27
4.4. Đặc tả bảng dữ liệu 27
4.5. Đặc tả các job Spark và yêu cầu hiệu năng 29
4.6. Đặc tả mô hình học máy 30
4.7. Đặc tả Delta Lake 30
4.8. Đặc tả ứng dụng web 31
4.9. Đặc tả Airflow 31
4.10. Rủi ro và phương án dự phòng 32
4.11. Danh mục nghiệm thu 32
ĐỀ TÀI 5 — THEO DÕI XU HƯỚNG VÀ CẢM XÚC TIN TỨC TIẾNG VIỆT 33
5.1. Bối cảnh nghiệp vụ 33
5.2. Nguồn dữ liệu 33
5.3. Kiến trúc hệ thống 33

5.4. Đặc tả bảng dữ liệu 34
5.5. Đặc tả các job Spark 36
5.6. Đặc tả mô hình học máy 36
5.7. Đặc tả Delta Lake 37
5.8. Đặc tả ứng dụng web 37
5.9. Đặc tả Airflow 38
5.10. Rủi ro và phương án dự phòng 39
5.11. Danh mục nghiệm thu 39
PHẦN C. PHỤ LỤC 40
C.1. Bảng so sánh 5 đề tài 40
C.2. Cấu hình Spark tham khảo cho môi trường hạn chế 40
C.3. Cấu trúc báo cáo cuối kỳ (15–20 trang) 40
C.4. Kịch bản demo 15 phút 41
C.5. Câu hỏi phản biện mẫu 41

PHẦN A. QUY ĐỊNH CHUNG

A.1. Mục tiêu môn học được đánh giá qua đồ án

Sau khi hoàn thành đồ án, sinh viên có khả năng:

Mã	Chuẩn đầu ra
CĐR1	Thiết kế và triển khai pipeline xử lý dữ liệu lớn theo kiến trúc Lakehouse (bronze–silver–gold)
CĐR2	Lập trình Spark ở cả chế độ batch và streaming; sử dụng Spark MLlib để huấn luyện và suy luận
CĐR3	Vận dụng Delta Lake: giao dịch ACID, MERGE/upsert, time travel, schema evolution, tối ưu file
CĐR4	Cấu hình và khai thác HDFS làm tầng lưu trữ phân tán
CĐR5	Điều phối pipeline tự động bằng Apache Airflow, xử lý phụ thuộc, retry, backfill
CĐR6	Xây dựng ứng dụng web trực quan hóa và tương tác với dữ liệu trong lakehouse
CĐR7	Làm việc nhóm, viết báo cáo kỹ thuật và bảo vệ giải pháp

A.2. Tổ chức nhóm và thời lượng

- Nhóm **3–4 sinh viên**. Nhóm phải nộp **bảng phân công vai trò**: Data Engineer, ML Engineer, Web Developer, DevOps/Orchestration. Mỗi thành viên phải có commit trong ít nhất 2 vai trò.
- Thời lượng: **8 tuần**, tính từ tuần 6 của học kỳ.
- Mỗi nhóm chọn **1 trong 5 đề tài**. Tối đa 2 nhóm/đề tài; nếu trùng, ưu tiên nhóm đăng ký trước.

A.3. Môi trường kỹ thuật bắt buộc

Thành phần	Phiên bản tối thiểu	Ghi chú
Apache Hadoop (HDFS + YARN)	3.3.x	Chạy pseudo-distributed hoặc cụm 3 node
Apache Spark	3.5.x	PySpark; submit lên YARN hoặc Spark Standalone
Delta Lake	3.1.x (`delta-spark`)	Tương thích Spark 3.5
Apache Airflow	2.9.x	`SparkSubmitOperator` hoặc `BashOperator`
Apache Kafka	3.6.x	Chỉ bắt buộc với đề tài có streaming
Web backend	FastAPI hoặc Flask	Streamlit được chấp nhận nhưng trừ 20% điểm phần web
Web frontend	React/Vue hoặc Jinja2 + Chart.js	
Quản lý mã nguồn	Git (GitHub/GitLab)	Bắt buộc, lịch sử commit là căn cứ chấm điểm cá nhân

Khuyến nghị đóng gói toàn bộ bằng `docker-compose` để tái lập môi trường. Nhóm không đủ tài nguyên có thể dùng 1 máy 16 GB RAM với cấu hình rút gọn (`spark.executor.memory=2g`, 2 executor).

A.4. Cấu trúc thư mục HDFS chuẩn

Mọi đề tài dùng chung quy ước:

```
hdfs:///lakehouse/<ma_de_tai>/
├── bronze/          # dữ liệu thô, append-only, giữ nguyên bản gốc
├── silver/          # đã làm sạch, chuẩn hóa, khử trùng lặp
├── gold/            # bảng tổng hợp phục vụ web app và báo cáo
├── features/        # bảng đặc trưng cho huấn luyện
├── models/          # artifact mô hình MLlib theo phiên bản
├── checkpoints/     # checkpoint của Structured Streaming
└── _quarantine/     # bản ghi lỗi bị loại khỏi silver
```

A.5. Cấu trúc repository chuẩn

```
<ma_de_tai>/
├── docker-compose.yml
├── conf/                    # spark-defaults.conf, hdfs-site.xml, .env.example
├── spark_jobs/
│   ├── ingest/              # nạp bronze
│   ├── transform/           # bronze→silver→gold
│   ├── streaming/           # job Structured Streaming
│   └── ml/                  # train, evaluate, predict
├── dags/                    # Airflow DAG
├── webapp/
│   ├── backend/
│   └── frontend/
├── notebooks/               # khảo sát dữ liệu (EDA)
├── tests/                   # kiểm thử chất lượng dữ liệu
├── docs/                    # sơ đồ kiến trúc, báo cáo
└── README.md                # hướng dẫn dựng lại hệ thống từ đầu
```

A.6. Yêu cầu bắt buộc chung (thiếu bất kỳ mục nào → không đạt)

1. **Spark**: tối thiểu 1 job batch và 1 mô hình MLlib. Đề tài có streaming phải có job Structured Streaming chạy liên tục ≥ 30 phút trong buổi demo.
2. **Delta Lake**: phải chứng minh được `MERGE INTO`, `DESCRIBE HISTORY`, truy vấn `VERSION AS OF`, và `OPTIMIZE` (kèm số liệu trước/sau).
3. **HDFS**: toàn bộ Delta table nằm trên HDFS, không được để ở local filesystem. Nộp kèm output `hdfs dfs -du -h /lakehouse/<ma_de_tai>`.
4. **Web**: tối thiểu 3 màn hình chức năng, có ít nhất 1 thao tác **ghi ngược** dữ liệu vào Delta table.
5. **Airflow**: tối thiểu 3 DAG với lịch khác nhau, có `retries`, `sla`, và ít nhất 1 nhánh rẽ (`BranchPythonOperator`) hoặc `Sensor`.

A.7. Mốc tiến độ

Tuần	Sản phẩm nộp	Trọng số
1	Đăng ký đề tài, phân công vai trò, sơ đồ kiến trúc dự kiến	5%
2	Dựng xong Hadoop + Spark + Delta; nạp được bronze; notebook EDA	10%
4	Hoàn thành silver + gold; DAG Airflow đầu tiên chạy được	15%
6	Mô hình MLlib có kết quả; web app hiển thị được dữ liệu gold	20%
8	Nộp toàn bộ + báo cáo + demo trực tiếp	50%

A.8. Thang điểm chi tiết

Hạng mục	Điểm	Tiêu chí chính
Spark (batch + MLlib/Streaming)	25	Đúng đắn, tận dụng xử lý phân tán, không kéo dữ liệu về driver
Delta Lake	20	MERGE, time travel, schema evolution, OPTIMIZE có đo lường
HDFS & hiệu năng	15	Phân vùng hợp lý, xử lý small files, số liệu benchmark
Ứng dụng web	15	Đủ màn hình, có ghi ngược, phản hồi < 3s
Airflow	15	DAG rõ ràng, xử lý lỗi, backfill được
Báo cáo & bảo vệ	10	Trình bày mạch lạc, trả lời được câu hỏi phản biện

Điểm cá nhân = điểm nhóm × hệ số đóng góp (0.7–1.2), căn cứ vào lịch sử Git và phần trả lời khi bảo vệ.

A.9. Chống sao chép

Nhóm phải giải thích được mọi dòng mã. Mã sinh bằng AI được phép nhưng phải ghi rõ trong `docs/AI_USAGE.md`: prompt đã dùng, phần nào do AI sinh, phần nào nhóm sửa. Không khai báo mà bị phát hiện → điểm 0 cho hạng mục liên quan.

PHẦN B. ĐỀ CƯƠNG CHI TIẾT TỪNG ĐỀ TÀI

ĐỀ TÀI 1 — HỆ THỐNG GIÁM SÁT VÀ DỰ BÁO PHỤ TẢI ĐIỆN

Mã đề tài: `bd01_dien_luc` **Độ khó:** Trung bình **Trọng tâm kỹ thuật:** Structured Streaming + hồi quy chuỗi thời gian + MERGE xử lý dữ liệu đến trễ

1.1. Bối cảnh nghiệp vụ

Một công ty điện lực quản lý hệ thống công tơ điện tử đọc chỉ số tự động theo chu kỳ 15 phút cho khoảng 5.000 khách hàng. Hai bài toán vận hành:

- **Dự báo phụ tải**: dự báo tổng công suất tiêu thụ 24 giờ tới (96 bước 15 phút) để lập kế hoạch mua điện và điều độ.
- **Phát hiện bất thường**: phát hiện khách hàng có mẫu tiêu thụ bất thường — nghi ngờ can thiệp công tơ, công tơ hỏng, hoặc rò rỉ điện.

Thực tế vận hành có ba khó khăn mà hệ thống phải xử lý: dữ liệu đến trễ (công tơ mất sóng, gửi bù sau vài giờ), dữ liệu bị chỉnh sửa (nhân viên hiệu chỉnh chỉ số sai), và dữ liệu thiếu (mất kết nối cả ngày).

1.2. Nguồn dữ liệu

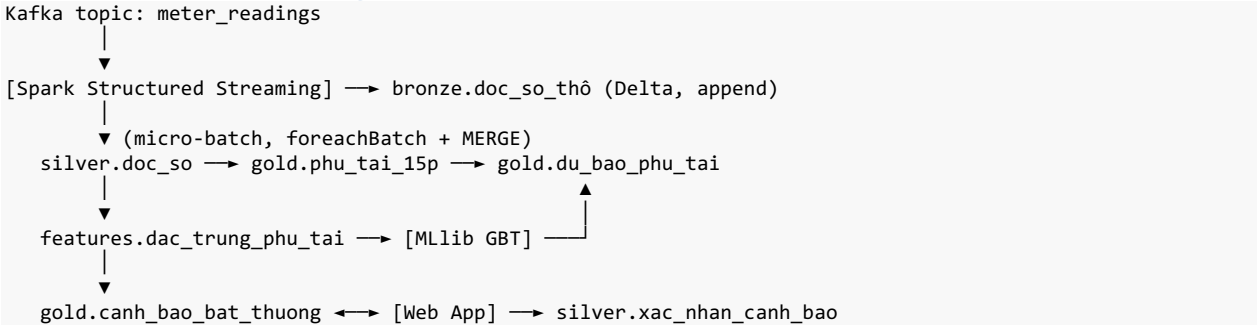
Chính: UCI *ElectricityLoadDiagrams20112014* — 370 khách hàng, chu kỳ 15 phút, 2011–2014, khoảng 140 triệu điểm đo. `https://archive.ics.uci.edu/dataset/321/electricityloaddiagrams20112014`

Bổ sung bắt buộc: nhóm viết script sinh dữ liệu mô phỏng để tạo ra 3 tình huống trên: - 5% bản ghi đến trễ 1–6 giờ - 1% bản ghi được gửi lại với giá trị đã hiệu chỉnh (cùng khóa, khác giá trị) - 2% khách hàng có bất thường được cấy chủ ý (giảm đột ngột 40% kéo dài, hoặc tăng vọt về đêm)

Dữ liệu tham chiếu: lịch ngày lễ Việt Nam, nhiệt độ theo giờ (Open-Meteo API lịch sử).

Bộ phát Kafka: script `producer.py` phát lại dữ liệu lịch sử với hệ số nén thời gian (1 ngày thực = 1 phút demo), có tham số `--late-rate`, `--correction-rate`.

1.3. Kiến trúc hệ thống



1.4. Đặc tả bảng dữ liệu

bronze.doc_so_tho

Ghi nguyên trạng, không kiểm tra. Phân vùng theo `ngay_nhan`.

Cột	Kiểu	Mô tả
`ma_khach_hang`	STRING	Mã công tơ
`thoi_diem_doc`	TIMESTAMP	Thời điểm đo (theo công tơ)
`cong_suat_kw`	DOUBLE	Công suất trung bình 15 phút
`thoi_diem_nhan`	TIMESTAMP	Thời điểm hệ thống nhận
`nguon`	STRING	`realtime` | `bu_tre` | `hieu_chinh`
`ngay_nhan`	DATE	Cột phân vùng

silver.doc_so

Khóa nghiệp vụ: `(ma_khach_hang, thoi_diem_doc)`. Cập nhật bằng `MERGE INTO`.

Cột	Kiểu	Mô tả
`ma_khach_hang`	STRING	
`thoi_diem_doc`	TIMESTAMP	
`cong_suat_kw`	DOUBLE	Giá trị sau hiệu chỉnh
`co_hieu_chinh`	BOOLEAN	Đã bị ghi đè hay chưa
`do_tre_phut`	INT	`thoi_diem_nhan - thoi_diem_doc`
`chat_luong`	STRING	`tot` | `noi_suy` | `nghi_ngo`
`ngay`	DATE	Cột phân vùng

Quy tắc làm sạch: - Loại bản ghi có `cong_suat_kw < 0` hoặc `> 3 × P95` của khách hàng đó → đưa vào `_quarantine` - Khoảng trống ≤ 4 bước (1 giờ) → nội suy tuyến tính, đánh dấu `chat_luong = 'noi_suy'` - Khoảng trống > 4 bước → giữ NULL, không nội suy

gold.phu_tai_15p

Tổng hợp toàn hệ thống, dùng cho dự báo tổng và cho biểu đồ web.

Cột	Kiểu
`thoi_diem`	TIMESTAMP
`tong_cong_suat_kw`	DOUBLE
`so_khach_hang_co_du_lieu`	INT
`ty_le_noi_suy`	DOUBLE

features.dac_trung_phu_tai

Bảng đặc trưng cho MLlib. Sinh lại mỗi khi silver thay đổi.

Nhóm đặc trưng	Chi tiết
Trễ (lag)	`lag_1`, `lag_2`, `lag_4`, `lag_96` (cùng giờ hôm trước), `lag_672` (cùng giờ tuần trước)
Thống kê trượt	trung bình/độ lệch chuẩn/max của 4, 96, 672 bước gần nhất
Thời gian	giờ, phút-trong-giờ, thứ trong tuần, tháng, `la_cuoi_tuan`, `la_ngay_le`
Chu kỳ	`sin(2πh/24)`, `cos(2πh/24)`, `sin(2πd/7)`, `cos(2πd/7)`
Thời tiết	nhiệt độ, độ ẩm tại `thoi_diem`, và trung bình 24h trước

gold.du_bao_phu_tai

Cột	Kiểu	Mô tả
`thoi_diem_du_bao`	TIMESTAMP	Thời điểm chạy dự báo
`thoi_diem_muc_tieu`	TIMESTAMP	Thời điểm được dự báo
`buoc`	INT	1..96
`gia_tri_du_bao`	DOUBLE	
`can_duoi`, `can_tren`	DOUBLE	Khoảng tin cậy từ phân vị residual
`phien_ban_mo_hinh`	STRING	Ví dụ `gbt_v20260315`

gold.canh_bao_bat_thuong

Cột	Kiểu	Mô tả
`ma_canh_bao`	STRING	UUID
`ma_khach_hang`	STRING	
`ngay_phat_hien`	DATE	
`loai`	STRING	`sut_giam` | `tang_vot` | `lech_cum` | `mat_tin_hieu`
`diem_bat_thuong`	DOUBLE	0–1
`bang_chung`	STRING (JSON)	Số liệu hỗ trợ để hiển thị trên web
`trang_thai`	STRING	`moi` | `dang_kiem_tra` | `xac_nhan` | `bo_qua`

silver.xac_nhan_canh_bao

Ghi ngược từ web app — đây là bảng chứng minh yêu cầu A.6.4.

Cột	Kiểu
`ma_canh_bao`	STRING
`nguoi_xu_ly`	STRING
`thoi_diem`	TIMESTAMP
`ket_luan`	STRING
`ghi_chu`	STRING

1.5. Đặc tả các job Spark

Job	Loại	Mô tả

Job	Loại	Mô tả
`streaming/nap_doc_so.py`	Streaming	Đọc Kafka → parse JSON → validate schema → ghi bronze; `trigger(processingTime='30 seconds')`, checkpoint trên HDFS
`transform/bronze_to_silver.py`	Batch/foreachBatch	Làm sạch, nội suy, `MERGE INTO silver.doc_so` theo khóa nghiệp vụ; bản ghi `hieu_chinh` ghi đè bản ghi cũ và đặt `co_hieu_chinh = true`
`transform/tao_gold.py`	Batch	Tổng hợp `gold.phu_tai_15p`; chỉ tính lại các ngày bị ảnh hưởng (đọc từ Delta CDF của silver)
`ml/tao_dac_trung.py`	Batch	Sinh `features.dac_trung_phu_tai` bằng Window functions
`ml/huan_luyen_du_bao.py`	Batch	Huấn luyện `GBTRegressor`, lưu `PipelineModel` vào `models/`
`ml/du_bao.py`	Batch	Sinh dự báo 96 bước, ghi `gold.du_bao_phu_tai`
`ml/phat_hien_bat_thuong.py`	Batch	Phát hiện bất thường cấp khách hàng

Ràng buộc lập trình: cấm dùng `.collect()`, `.toPandas()` trên DataFrame chưa được lọc/tổng hợp xuống dưới 10.000 dòng. Vi phạm bị trừ điểm hạng mục Spark.

1.6. Đặc tả mô hình học máy

Mô hình A — Dự báo phụ tải

- **Thuật toán:** `GBTRegressor` (MLlib). Bắt buộc so sánh với 2 baseline: naive (giá trị cùng giờ hôm trước) và `LinearRegression`.
- **Chiến lược đa bước:** direct multi-step — huấn luyện riêng cho các horizon nhóm {1–4, 5–24, 25–96}, hoặc recursive. Nhóm phải giải thích lựa chọn.
- **Chia dữ liệu:** time-based split, không shuffle. Train: 2011–2013. Validation: 6 tháng đầu 2014. Test: 6 tháng cuối 2014.
- **Metric:** MAPE (chính), RMSE, MAE. Báo cáo theo từng horizon.
- **Ngưỡng đạt:** MAPE ≤ 8% cho horizon 1–4 bước và ≤ 15% cho horizon 96 bước trên tập test; đồng thời phải **tốt hơn baseline naive ít nhất 20%**.
- **Tuning:** `CrossValidator` với `TimeSeriesSplit` tự cài (MLlib không có sẵn — nhóm tự viết bằng cách tạo folds theo thời gian), lưới tham số tối thiểu: `maxDepth ∈ {5, 8}`, `maxIter ∈ {50, 100}`, `stepSize ∈ {0.05, 0.1}`.

Mô hình B — Phát hiện bất thường

Kết hợp 3 tín hiệu, mỗi tín hiệu cho điểm 0–1, điểm cuối là max:

1. **Lệch residual**: huấn luyện mô hình dự báo riêng cho từng khách hàng (hoặc mô hình chung có `ma_khach_hang` là feature); residual chuẩn hóa vượt 3σ liên tục ≥ 8 bước → `sut_giam`/`tang_vot`.
2. **Lệch cụm**: `KMeans` (k=5..8, chọn bằng silhouette) trên vector hồ sơ tiêu thụ 24h chuẩn hóa; khách hàng có khoảng cách tới tâm cụm vượt P99 → `lech_cum`.
3. **Mất tín hiệu**: không có dữ liệu > 6 giờ trong giờ hành chính.

Đánh giá: vì dữ liệu thật không có nhãn, nhóm dùng các bất thường **đã cấy chủ ý** ở mục 1.2 làm ground truth. Báo cáo Precision, Recall, F1. **Ngưỡng đạt:** Recall ≥ 0.80 với Precision ≥ 0.50.

1.7. Đặc tả Delta Lake

Yêu cầu	Cách thực hiện trong đề tài này
MERGE / upsert	`silver.doc_so`: `WHEN MATCHED AND source.nguon = 'hieu_chinh' THEN UPDATE`, `WHEN NOT MATCHED THEN INSERT`
Time travel	So sánh dự báo sinh từ dữ liệu **trước** khi bản ghi trễ về vs **sau** khi về: `SELECT … FROM silver.doc_so VERSION AS OF <v>`. Web app có màn hình "Ảnh hưởng của dữ liệu đến trễ" dùng chính chức năng này
Schema evolution	Tuần 6 bổ sung cột `dien_ap_v` và `he_so_cong_suat` vào bronze bằng `mergeSchema = true`; báo cáo phải chụp lại `DESCRIBE HISTORY` trước/sau
Change Data Feed	Bật `delta.enableChangeDataFeed = true` trên `silver.doc_so`; job `tao_gold.py` chỉ đọc phần thay đổi thay vì quét toàn bảng
OPTIMIZE / Z-ORDER	`OPTIMIZE silver.doc_so WHERE ngay >= ... ZORDER BY (ma_khach_hang)`. Đo và báo cáo: số file, kích thước file trung bình, thời gian truy vấn mẫu trước/sau
VACUUM	DAG hằng tuần, `RETAIN 168 HOURS`

Bẫy kỹ thuật cần xử lý: streaming ghi vào bronze mỗi 30 giây sinh rất nhiều file nhỏ. Nhóm phải đo hiện tượng này và trình bày giải pháp (auto-compaction hoặc DAG `OPTIMIZE` định kỳ).

1.8. Đặc tả ứng dụng web

Màn hình	Nội dung	API
Tổng quan	Biểu đồ phụ tải thực tế 7 ngày + dải dự báo 24h tới có khoảng tin cậy; KPI: MAPE 7 ngày qua, số khách hàng mất tín hiệu	`GET /api/phu-tai?tu=&den=`, `GET /api/du-bao`
Cảnh báo	Bảng cảnh báo lọc theo loại/trạng thái/điểm; nhấn vào 1 dòng mở panel chi tiết có biểu đồ tiêu thụ của khách hàng đó và đường bao "bình thường"	`GET /api/canh-bao`, `GET /api/khach-hang/{id}/tieu-thu`
Xử lý cảnh báo	Nút *Xác nhận vi phạm* / *Bỏ qua* + ô ghi chú → **ghi vào** `silver.xac_nhan_canh_bao` và cập nhật `trang_thai` trong `gold.canh_bao_bat_thuong` bằng MERGE	`POST /api/canh-bao/{id}/ket-luan`
Ảnh hưởng dữ liệu trễ	Chọn 1 ngày, hệ thống hiển thị dự báo tại các phiên bản Delta khác nhau của ngày đó và mức sai lệch	`GET /api/time-travel?ngay=&phien_ban=`
Quản trị	Bảng lịch sử chạy DAG (gọi Airflow REST API), nút chạy lại dự báo thủ công	`POST /api/dag/{dag_id}/trigger`

Yêu cầu hiệu năng: mọi màn hình phản hồi < 3 giây. Web **không** được query trực tiếp Delta table lớn cho mỗi request — phải đọc từ bảng gold đã tổng hợp, hoặc dùng lớp cache/DuckDB đọc Parquet đã lọc.

1.9. Đặc tả Airflow

DAG 1 — `dl_pipeline_hang_gio` (`0 * * * *`)

```
kiem_tra_du_lieu_moi (Sensor)
        │
        ▼
kiem_tra_chat_luong (PythonOperator)
        │
        ├──[BranchPythonOperator]──► canh_bao_chat_luong (EmailOperator) ──► ket_thuc
        │
        ▼ (đạt)
bronze_to_silver (SparkSubmitOperator)
        │
        ▼
tao_gold ──► tao_dac_trung ──► du_bao ──► cap_nhat_cache_web
```

- `retries=2`, `retry_delay=5 phút`, `sla=30 phút`
- Nhánh rẽ dựa trên: tỷ lệ bản ghi vào `_quarantine` > 5% hoặc số khách hàng có dữ liệu < 80%

DAG 2 — `dl_huan_luyen_lai` (`0 2 * * 0` — 2h sáng Chủ nhật)

```
chuan_bi_du_lieu ──► huan_luyen ──► danh_gia
                                        │
                          ┌─────────────┴─────────────┐
                          ▼                           ▼
                promote_mo_hinh              giu_mo_hinh_cu + canh_bao
            (MAPE mới < MAPE cũ)             (MAPE mới ≥ MAPE cũ)
```

Model registry đơn giản: bảng Delta `models/registry` ghi `phien_ban`, `duong_dan`, `mape_test`, `dang_su_dung`.

DAG 3 — `dl_bao_tri` (`0 3 * * *`)

`OPTIMIZE` các bảng → `VACUUM` → thống kê dung lượng HDFS → ghi vào `gold.thong_ke_he_thong`.

DAG 4 — `dl_backfill` (không lịch, chạy thủ công)

Nhận `start_date`, `end_date` qua `dag_run.conf`, chạy lại toàn bộ pipeline cho khoảng ngày đó. **Bắt buộc demo backfill 7 ngày trong buổi bảo vệ.**

1.10. Rủi ro và phương án dự phòng

Rủi ro	Dấu hiệu	Phương án
Không đủ RAM chạy 370 khách hàng	OOM trong huấn luyện	Giảm còn 100 khách hàng, giữ nguyên logic; ghi rõ trong báo cáo
Kafka không ổn định	Consumer lag tăng liên tục	Thay bằng nguồn file streaming (`readStream.format("json")` trên thư mục HDFS được đổ file định kỳ) — được chấp nhận, trừ 10% điểm streaming
MAPE không đạt ngưỡng	Mô hình kém baseline	Vẫn nộp, nhưng phải có phần **phân tích nguyên nhân** với ít nhất 3 giả thuyết được kiểm chứng bằng số liệu
Không kịp làm web đầy đủ	Tuần 6 chưa có UI	Ưu tiên màn hình Tổng quan + Cảnh báo + ghi ngược; bỏ màn hình Time travel

1.11. Danh mục nghiệm thu

- ☐ `hdfs dfs -ls -R /lakehouse/bd01_dien_luc` hiển thị đủ 7 thư mục
- ☐ Streaming job chạy liên tục ≥ 30 phút, không tăng lag
- ☐ `DESCRIBE HISTORY silver.doc_so` có ít nhất 1 thao tác MERGE và 1 lần schema evolution
- ☐ Truy vấn `VERSION AS OF` cho ra kết quả khác nhau và giải thích được vì sao
- ☐ Bảng so sánh MAPE: naive / LinearRegression / GBT theo 3 nhóm horizon
- ☐ Precision/Recall phát hiện bất thường trên tập bất thường đã cấy
- ☐ Số liệu OPTIMIZE trước/sau (số file, thời gian query)
- ☐ Demo: bấm nút trên web → cảnh báo đổi trạng thái → `SELECT` từ Delta thấy bản ghi mới
- ☐ Demo: trigger DAG backfill 7 ngày, xem log Airflow

ĐỀ TÀI 2 — PHÁT HIỆN GIAN LẬN GIAO DỊCH THẺ THEO THỜI GIAN THỰC

Mã đề tài: `bd02_gian_lan` **Độ khó:** Cao **Trọng tâm kỹ thuật:** Structured Streaming có trạng thái + phân loại mất cân bằng + vòng phản hồi nhãn người dùng

2.1. Bối cảnh nghiệp vụ

Trung tâm thẻ của một ngân hàng nhận trung bình 300 giao dịch/giây vào giờ cao điểm. Yêu cầu:

- Chấm điểm rủi ro mỗi giao dịch trong **dưới 2 giây** kể từ khi giao dịch phát sinh.
- Giao dịch điểm cao được đẩy vào hàng đợi cho thẩm định viên xem xét; kết luận của thẩm định viên trở thành nhãn huấn luyện cho chu kỳ sau.
- Mô hình phải chịu được **concept drift**: thủ đoạn gian lận thay đổi liên tục.

Ràng buộc kinh doanh quan trọng: đội thẩm định chỉ xử lý được **200 giao dịch/ngày**. Vì vậy mục tiêu không phải là "recall cao nhất" mà là **tối đa hóa số giao dịch gian lận bắt được trong 200 cảnh báo hàng đầu** — đây là bài toán Precision@K, không phải accuracy.

2.2. Nguồn dữ liệu

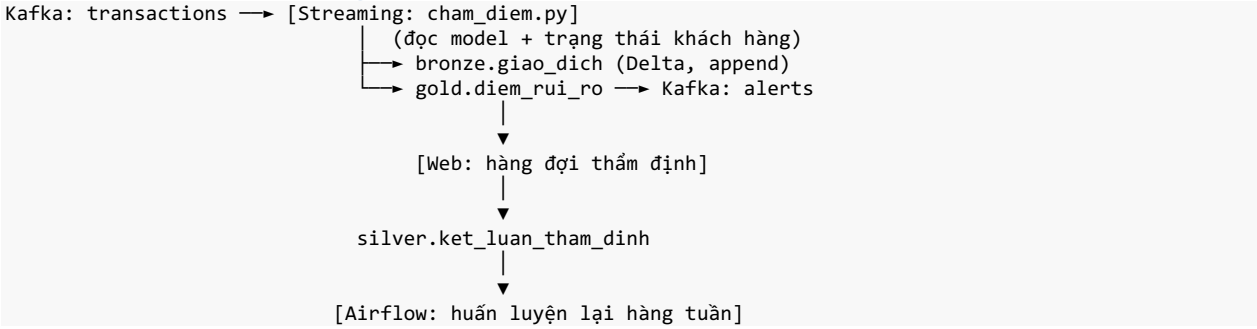
Chính: *PaySim* — mô phỏng giao dịch chuyển tiền di động, 6,3 triệu giao dịch, có cột nhãn `isFraud`. `https://www.kaggle.com/datasets/ealaxi/paysim1`

Thay thế / bổ sung: *Credit Card Fraud Detection* (ULB) — 284.807 giao dịch, tỷ lệ gian lận 0,172%, đặc trưng đã PCA hóa (V1–V28). `https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud`

Khuyến nghị: dùng PaySim làm chính (có `nameOrig`, `nameDest`, `type` — cho phép xây đặc trưng hành vi thật sự), dùng bộ ULB để đối chiếu.

Bộ phát Kafka: phát lại theo `step` (mỗi step = 1 giờ mô phỏng), nén còn 10 giây thực. Bắt buộc mô phỏng **giao dịch lệch thứ tự** (out-of-order) tới 30 giây để buộc nhóm dùng watermark.

2.3. Kiến trúc hệ thống



2.4. Đặc tả bảng dữ liệu

bronze.giao_dich

Cột	Kiểu	Mô tả
`ma_giao_dich`	STRING	

Cột	Kiểu	Mô tả
thoi_diem	TIMESTAMP	Thời điểm giao dịch
thoi_diem_nhan	TIMESTAMP	Thời điểm hệ thống nhận
tai_khoan_nguon	STRING	
tai_khoan_dich	STRING	
loai	STRING	CHUYEN_KHOAN | RUT_TIEN | THANH_TOAN | NAP_TIEN
so_tien	DOUBLE	
so_du_truoc_nguon, so_du_sau_nguon	DOUBLE	
so_du_truoc_dich, so_du_sau_dich	DOUBLE	
kenh	STRING	ATM | POS | ONLINE | APP
ngay	DATE	Cột phân vùng

silver.trang_thai_khach_hang

Bảng trạng thái cập nhật liên tục bằng MERGE từ streaming — đây là **đặc trưng lịch sử** dùng cho suy luận.

Cột	Kiểu	Mô tả
tai_khoan	STRING	Khóa
so_gd_1h, so_gd_24h, so_gd_7d	INT	Đếm theo cửa sổ
tong_tien_24h	DOUBLE	
so_tien_tb_30d, so_tien_dolech_30d	DOUBLE	Dùng để tính z-score
so_tk_dich_khac_nhau_24h	INT	Dấu hiệu phát tán tiền
gio_giao_dich_pho_bien	ARRAY	Top 3 khung giờ thường giao dịch
lan_gd_gan_nhat	TIMESTAMP	
cap_nhat_luc	TIMESTAMP	

gold.diem_rui_ro

Cột	Kiểu
ma_giao_dich	STRING
diem_rui_ro	DOUBLE (0–1)
xep_hang_trong_ngay	INT
cac_yeu_to_chinh	ARRAY<STRUCT<ten:STRING, dong_gop:DOUBLE>>
phien_ban_mo_hinh	STRING
do_tre_ms	LONG
trang_thai_tham_dinh	STRING

silver.ket_luan_tham_dinh

Cột	Kiểu

Cột	Kiểu
ma_giao_dich	STRING
nhan	INT (0/1)
nguoi_tham_dinh	STRING
thoi_diem_ket_luan	TIMESTAMP
ly_do	STRING
thoi_gian_xu_ly_giay	INT

gold.theo_doi_drift

Cột	Kiểu
ngay	DATE
ten_dac_trung	STRING
psi	DOUBLE
ks_statistic	DOUBLE
muc_do	STRING (on_dinh/canh_bao/nghiem_trong)

2.5. Đặc tả các job Spark

Job	Loại	Yêu cầu kỹ thuật
streaming/cham_diem.py	Streaming	withWatermark("thoi_diem", "1 minute"); join stream với silver.trang_thai_khach_hang (stream-static join); load PipelineModel trong foreachBatch; ghi 2 sink
streaming/cap_nhat_trang_thai.py	Streaming stateful	flatMapGroupsWithState theo tai_khoan để duy trì bộ đếm cửa sổ trượt; GroupStateTimeout.ProcessingTimeTimeout dọn state khách hàng không hoạt động 30 ngày
transform/bronze_to_silver.py	Batch	Chuẩn hóa, phát hiện bản ghi trùng, ghi _quarantine
ml/tao_tap_huan_luyen.py	Batch	Ghép giao dịch + trạng thái tại thời điểm giao dịch (**point-in-time join** — chống data leakage) + nhãn từ thẩm định viên
ml/huan_luyen.py	Batch	Huấn luyện, tuning, đánh giá, đăng ký mô hình
ml/kiem_tra_drift.py	Batch	Tính PSI và KS giữa cửa sổ 7 ngày gần nhất và tập huấn luyện

Điểm khó bắt buộc phải xử lý — data leakage: khi tạo tập huấn luyện, đặc trưng của một giao dịch phải là trạng thái khách hàng **tại thời điểm trước giao dịch đó**, không phải trạng thái hiện tại. Nhóm phải cài point-in-time join (tái tính đặc trưng bằng Window function theo thời gian, hoặc lưu snapshot trạng thái theo phiên bản Delta). Đây là hạng mục chấm điểm riêng.

2.6. Đặc tả mô hình học máy

- **Thuật toán chính:** GBTClassifier. So sánh bắt buộc với RandomForestClassifier và LogisticRegression (có regularization).
- **Xử lý mất cân bằng:** thử ít nhất 2 trong 3 cách và báo cáo so sánh:

1. `weightCol` — trọng số lớp nghịch đảo tần suất
2. Undersampling lớp âm về tỷ lệ 1:20
3. Điều chỉnh ngưỡng quyết định theo đường cong Precision–Recall (không dùng 0.5)

- **Chia dữ liệu:** theo thời gian. Train 70% đầu, validation 15%, test 15% cuối. **Tuyệt đối không random split** — sẽ bị trừ điểm nặng vì rò rỉ thời gian.
- **Metric chính: Precision@200/ngày** (số gian lận thật trong 200 cảnh báo điểm cao nhất). Metric phụ: PR-AUC, ROC-AUC, Recall@Precision=0.5.
- **Ngưỡng đạt:** Precision@200 ≥ 0.35 và PR-AUC ≥ 0.60 trên tập test. Nếu dùng bộ ULB: PR-AUC ≥ 0.75.
- **Giải thích mô hình:** trích `featureImportances` của GBT; với mỗi cảnh báo, hiển thị top 3 đặc trưng đóng góp (dùng chênh lệch so với trung bình khách hàng — không bắt buộc SHAP).

Vòng phản hồi: mỗi tuần, tập huấn luyện được bổ sung nhãn từ `silver.ket_luan_tham_dinh`. Báo cáo phải có biểu đồ Precision@200 theo tuần để cho thấy vòng phản hồi có (hoặc không) cải thiện mô hình.

2.7. Đặc tả Delta Lake

Yêu cầu	Cách thực hiện
MERGE	`silver.trang_thai_khach_hang` cập nhật liên tục từ streaming bằng `foreachBatch` + `MERGE`; `gold.diem_rui_ro` cập nhật `trang_thai_tham_dinh` khi thẩm định viên kết luận
Time travel	Tái lập **chính xác** tập huấn luyện của một phiên bản mô hình: `SELECT ... VERSION AS OF <v_luc_huan_luyen>`. Đây là yêu cầu kiểm toán thật trong ngành ngân hàng — nhóm phải chứng minh tái lập được kết quả cũ
Change Data Feed	Web app đọc CDF của `gold.diem_rui_ro` để cập nhật hàng đợi cảnh báo mà không quét lại toàn bảng
Schema evolution	Tuần 5 thêm cột `ma_thiet_bi` và `dia_chi_ip` vào bronze; mô hình phiên bản sau dùng thêm đặc trưng "thiết bị lạ"
OPTIMIZE / Z-ORDER	`ZORDER BY (tai_khoan_nguon, thoi_diem)`; đo thời gian truy vấn "toàn bộ giao dịch của 1 tài khoản trong 30 ngày" trước/sau
Constraints	`ALTER TABLE ... ADD CONSTRAINT so_tien_duong CHECK (so_tien > 0)` — chứng minh Delta từ chối ghi dữ liệu vi phạm

2.8. Đặc tả ứng dụng web

Màn hình	Nội dung	API
Hàng đợi cảnh báo	Danh sách cuộn theo thời gian thực (WebSocket hoặc polling 5s), sắp xếp theo điểm rủi ro; badge hiển thị số cảnh báo chưa xử lý	`GET /api/canh-bao?trang_thai=moi`
Chi tiết giao dịch	Thông tin giao dịch, top 3 yếu tố rủi ro dạng thanh ngang, lịch sử 20 giao dịch gần nhất của tài khoản, biểu đồ số tiền theo thời gian có đánh dấu giao dịch hiện tại	`GET /api/giao-dich/{id}`
Kết luận	Nút *Gian lận* / *Hợp lệ* + lý do (dropdown) → **ghi**	`POST /api/giao-dich/{id}/ket-luan`

Màn hình	Nội dung	API
thẩm định	`silver.ket_luan_tham_dinh`	
Bảng điều khiển mô hình	Precision@200 theo tuần, PR curve, biểu đồ PSI theo đặc trưng, lịch sử phiên bản mô hình	`GET /api/mo-hinh/hieu-nang`
Kiểm toán / Time travel	Chọn phiên bản mô hình → hiển thị phiên bản Delta của tập huấn luyện tương ứng và cho phép chạy lại đánh giá	`GET /api/kiem-toan/{phien_ban}`

Yêu cầu độ trễ: hiển thị cảnh báo trên web trong vòng **5 giây** kể từ khi giao dịch được phát ra Kafka. Nhóm phải đo và báo cáo phân phối độ trễ end-to-end (P50, P95, P99).

2.9. Đặc tả Airflow

DAG 1 — `gl_giam_sat_streaming` (`*/10 * * * *`)

Kiểm tra streaming job còn sống: đọc Spark REST API, kiểm tra consumer lag của Kafka, kiểm tra số bản ghi mới trong bronze 10 phút qua. Nếu bất thường → gửi cảnh báo và tự động restart job (`BashOperator` gọi `spark-submit`).

DAG 2 — `gl_kiem_tra_drift` (`0 1 * * *`)

```
tinh_psi_ks
     │
[BranchPythonOperator]
     ├── psi < 0.1   ──► ghi_nhat_ky
     ├── 0.1–0.25    ──► ghi_nhat_ky + canh_bao_email
     └── > 0.25      ──► kich_hoat_dag_huan_luyen (TriggerDagRunOperator)
```

DAG 3 — `gl_huan_luyen_lai` (`0 3 * * 1`)

```
kiem_tra_du_nhan (≥ 500 nhãn mới?)
     │
     ▼
tao_tap_huan_luyen (point-in-time join)
     │
     ▼
huan_luyen ──► danh_gia_test
     │
[so sánh Precision@200 với mô hình đang chạy]
     ├── tốt hơn ──► dang_ky_mo_hinh ──► nap_lai_streaming_job
     └── kém hơn ──► giu_nguyen + bao_cao
```

DAG 4 — `gl_bao_tri` (`30 4 * * *`)

`OPTIMIZE` + `ZORDER` + `VACUUM` + xóa state streaming của tài khoản không hoạt động.

2.10. Rủi ro và phương án dự phòng

Rủi ro	Phương án
Streaming stateful quá phức tạp	Cho phép thay `flatMapGroupsWithState` bằng cách join stream với bảng trạng thái được cập nhật theo micro-batch (đơn giản hơn, trừ 10% điểm streaming)
Không đạt Precision@200	Nộp kèm phân tích: đường cong Precision@K với K = 50, 100, 200, 500 và giải thích đánh đổi

Rủi ro	Phương án
Nhãn thẩm định viên quá ít để huấn luyện lại	Dùng nhãn gốc `isFraud` của PaySim làm nhãn "thẩm định viên hoàn hảo", sinh tự động cho các giao dịch được đưa vào hàng đợi
Độ trễ end-to-end > 5s	Giảm `trigger` xuống 2 giây, tăng số partition Kafka, giảm kích thước state

2.11. Danh mục nghiệm thu

- ☐ Streaming chạy ≥ 30 phút với tốc độ ≥ 100 giao dịch/giây, lag ổn định
- ☐ Biểu đồ phân phối độ trễ end-to-end (P50/P95/P99)
- ☐ Chứng minh point-in-time join không rò rỉ dữ liệu (trình bày logic + 1 ví dụ cụ thể)
- ☐ Bảng so sánh 3 thuật toán × 2 chiến lược mất cân bằng theo Precision@200 và PR-AUC
- ☐ Tái lập tập huấn luyện của mô hình cũ bằng `VERSION AS OF` và cho ra đúng metric đã ghi
- ☐ Delta constraint từ chối được bản ghi `so_tien <= 0` (demo trực tiếp)
- ☐ Demo: thẩm định viên bấm nút → nhãn xuất hiện trong Delta → DAG huấn luyện lại dùng được nhãn đó
- ☐ Demo DAG drift: cố tình đẩy dữ liệu lệch phân phối → DAG tự kích hoạt huấn luyện lại

ĐỀ TÀI 3 — HỆ KHUYẾN NGHỊ SẢN PHẨM CHO SÀN THƯƠNG MẠI ĐIỆN TỬ

Mã đề tài: `bd03_khuyen_nghi` **Độ khó:** Trung bình **Trọng tâm kỹ thuật:** MLlib ALS + kết hợp batch và streaming + thí nghiệm A/B tái lập bằng time travel

3.1. Bối cảnh nghiệp vụ

Một sàn thương mại điện tử có 1,4 triệu người dùng và 235.000 sản phẩm. Ban sản phẩm muốn:

- Gợi ý cá nhân hóa "Dành cho bạn" trên trang chủ.
- Gợi ý "Sản phẩm liên quan" trên trang chi tiết.
- Khối "Đang thịnh hành" cập nhật theo thời gian thực (cửa sổ 15 phút).
- Đo được khối gợi ý nào mang lại CTR cao hơn thông qua **thí nghiệm A/B**, và tái lập được kết quả thí nghiệm khi bị chất vấn.

Ba thách thức: **cold start** (người dùng và sản phẩm mới), **implicit feedback** (chỉ có click/xem/mua, không có rating), và **popularity bias** (mô hình chỉ gợi ý hàng bán chạy).

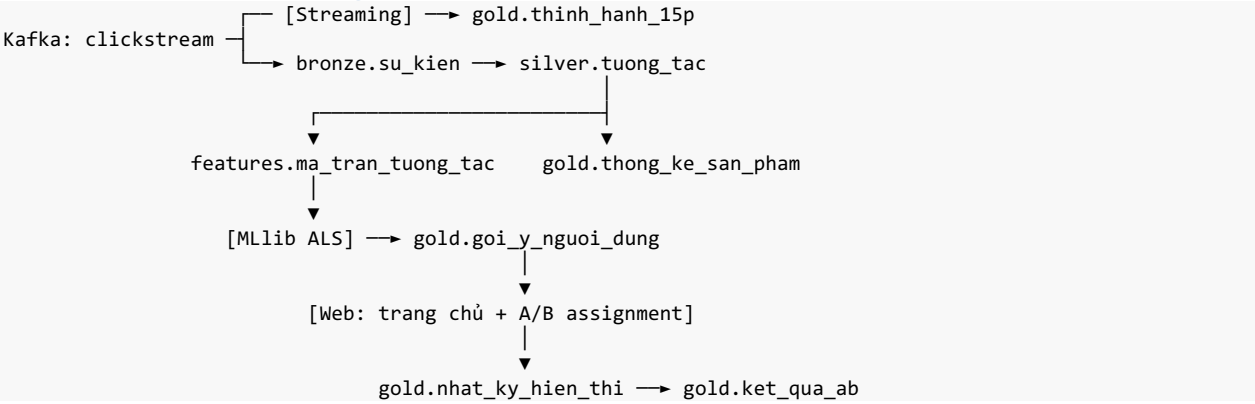
3.2. Nguồn dữ liệu

Chính: *RetailRocket Recommender System Dataset* — 2,7 triệu sự kiện (`view`, `addtocart`, `transaction`) của 1,4 triệu khách trong 4,5 tháng, kèm thuộc tính sản phẩm và cây danh mục.
`https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset`

Thay thế: *Amazon Reviews 2023* (một danh mục con, ví dụ Electronics) nếu cần dữ liệu có văn bản.

Bộ phát Kafka: phát lại sự kiện theo timestamp gốc, nén 1 ngày = 30 giây. Ngoài ra sinh thêm 5% người dùng mới hoàn toàn để buộc nhóm xử lý cold start.

3.3. Kiến trúc hệ thống



3.4. Đặc tả bảng dữ liệu

bronze.su_kien

Cột	Kiểu

Cột	Kiểu
ma_su_kien	STRING
thoi_diem	TIMESTAMP
ma_nguoi_dung	STRING
ma_phien	STRING
ma_san_pham	STRING
loai_su_kien	STRING (xem/them_gio/mua/bo_gio)
ma_danh_muc	STRING
thiet_bi	STRING
ngay	DATE (phân vùng)

silver.tuong_tac

Đã khử trùng lặp (cùng user–item–loại trong 5 giây), đã lọc bot (>500 sự kiện/giờ).

Cột	Kiểu	Mô tả
ma_nguoi_dung	STRING	
ma_san_pham	STRING	
thoi_diem	TIMESTAMP	
loai_su_kien	STRING	
trong_so	DOUBLE	xem=1.0, them_gio=3.0, mua=5.0
ngay	DATE	

features.ma_tran_tuong_tac

Bảng đầu vào cho ALS. Trọng số có **suy giảm theo thời gian**: `trong_so × exp(-λ × so_ngay_truoc)` với λ = 0,01.

Cột	Kiểu
user_idx	INT (StringIndexer)
item_idx	INT
diem	DOUBLE

gold.goi_y_nguoi_dung

Cột	Kiểu
ma_nguoi_dung	STRING
xep_hang	INT (1..20)
ma_san_pham	STRING
diem_du_doan	DOUBLE
nguon	STRING (als / pho_bien_danh_muc / thinh_hanh)

Cột	Kiểu
phien_ban_mo_hinh	STRING
sinh_luc	TIMESTAMP

gold.thinh_hanh_15p

Sinh bởi streaming, cửa sổ trượt 15 phút / bước 1 phút.

Cột	Kiểu
cua_so_bat_dau, cua_so_ket_thuc	TIMESTAMP
ma_san_pham	STRING
so_luot_xem, so_luot_them_gio	INT
diem_thinh_hanh	DOUBLE

`diem_thinh_hanh = (luot_xem_hien_tai + 3×them_gio) / (trung_binh_24h + 10)` — chuẩn hóa để sản phẩm ít phổ biến vẫn có cơ hội xuất hiện.

gold.nhat_ky_hien_thi (impression log)

Cột	Kiểu
ma_hien_thi	STRING
ma_nguoi_dung	STRING
nhom_ab	STRING (A/B)
vi_tri	INT
ma_san_pham	STRING
thoi_diem_hien_thi	TIMESTAMP
da_click	BOOLEAN
da_mua	BOOLEAN

gold.ket_qua_ab

Cột	Kiểu
ngay	DATE
nhom_ab	STRING
so_hien_thi, so_click, so_mua	LONG
ctr, ty_le_chuyen_doi	DOUBLE
p_value	DOUBLE
phien_ban_delta_du_lieu	LONG

3.5. Đặc tả các job Spark

Job	Loại	Yêu cầu
streaming/nap_su_kien.py	Streaming	Kafka → bronze, checkpoint HDFS

Job	Loại	Yêu cầu
`streaming/tinh_thinh_hanh.py`	Streaming	Cửa sổ trượt + watermark 2 phút; ghi `gold.thinh_hanh_15p` bằng `outputMode("update")` + MERGE
`transform/bronze_to_silver.py`	Batch	Khử trùng lặp, lọc bot, gán trọng số
`ml/chuan_bi_ma_tran.py`	Batch	`StringIndexer` cho user/item (lưu index để tái sử dụng), suy giảm thời gian
`ml/huan_luyen_als.py`	Batch	ALS `implicitPrefs=true`, tuning, sinh top-20 cho toàn bộ user bằng `recommendForAllUsers`
`ml/tron_goi_y.py`	Batch	Trộn ALS + thịnh hành + phổ biến theo danh mục; xử lý cold start
`transform/tinh_ket_qua_ab.py`	Batch	Tổng hợp impression log, kiểm định z-test hai tỷ lệ

3.6. Đặc tả mô hình học máy

- **Thuật toán:** `ALS` với `implicitPrefs = true`, `coldStartStrategy = "drop"` khi đánh giá.
- **Tuning bắt buộc:** `rank ∈ {10, 30, 50}`, `regParam ∈ {0.01, 0.1, 1.0}`, `alpha ∈ {1, 10, 40}`. Tối thiểu 9 tổ hợp.
- **Chia dữ liệu:** theo thời gian — train 4 tháng đầu, test tháng cuối. Với mỗi user trong test, giữ lại các item họ tương tác trong tháng cuối làm ground truth.
- **Metric:** Precision@10, Recall@10, NDCG@10, MAP@10. Bổ sung 2 metric ngoài độ chính xác:
  - **Coverage**: tỷ lệ sản phẩm trong catalog xuất hiện ở ít nhất 1 danh sách gợi ý
  - **Novelty**: độ phổ biến trung bình (nghịch đảo) của item được gợi ý — đo popularity bias
- **Baseline bắt buộc:** (1) gợi ý top sản phẩm bán chạy toàn sàn, (2) top bán chạy trong danh mục user hay xem.
- **Ngưỡng đạt:** NDCG@10 của ALS **cao hơn baseline bán chạy ít nhất 30%**, đồng thời Coverage ≥ 10%.

Xử lý cold start (bắt buộc, chấm điểm riêng)

Trường hợp	Chiến lược
User mới hoàn toàn	Top sản phẩm thịnh hành 15 phút gần nhất
User có < 5 tương tác	Top bán chạy trong các danh mục đã xem, trộn 50/50 với thịnh hành
Item mới	Không có trong ALS → chèn theo độ tương đồng danh mục vào 20% vị trí cuối danh sách

3.7. Đặc tả Delta Lake

Yêu cầu	Cách thực hiện
MERGE	`gold.goi_y_nguoi_dung` cập nhật theo user (không ghi đè toàn bảng); `gold.thinh_hanh_15p` upsert theo cửa sổ
Time travel	Mỗi lần huấn luyện ghi lại `phien_ban_delta` của `features.ma_tran_tuong_tac`. Khi

Yêu cầu	Cách thực hiện
(trọng tâm đề tài)	cần kiểm chứng kết quả A/B, chạy lại chính xác pipeline trên `VERSION AS OF` đó. Web có màn hình "Tái lập thí nghiệm"
Schema evolution	Tuần 5 thêm `gia_san_pham` và `xep_hang_sao` vào silver; mô hình v2 dùng thêm ràng buộc lọc theo tầm giá
Change Data Feed	Bật trên `gold.nhat_ky_hien_thi` để job tính A/B chỉ xử lý phần mới
OPTIMIZE / Z-ORDER	`ZORDER BY (ma_nguoi_dung)` trên `gold.goi_y_nguoi_dung` — đo thời gian lấy gợi ý của 1 user trước/sau (đây chính là truy vấn nóng của web)
Partition	`silver.tuong_tac` phân vùng theo `ngay`; giải thích vì sao **không** phân vùng theo `ma_nguoi_dung` (quá nhiều partition nhỏ)

3.8. Đặc tả ứng dụng web

Web app đóng vai trò **giả lập sàn TMĐT** để sinh impression log thật.

Màn hình	Nội dung	API
Trang chủ (giả lập)	Chọn user từ dropdown (mô phỏng đăng nhập); hiển thị khối "Dành cho bạn" 10 sản phẩm + khối "Đang thịnh hành"; mỗi lần render **ghi impression log**	`GET /api/goi-y/{user_id}`, `POST /api/hien-thi`
Trang sản phẩm	Chi tiết sản phẩm + khối "Sản phẩm liên quan"; nút *Xem*, *Thêm giỏ*, *Mua* → **đẩy sự kiện vào Kafka** (khép kín vòng dữ liệu)	`POST /api/su-kien`
Bảng điều khiển A/B	CTR và tỷ lệ chuyển đổi theo nhóm A/B, biểu đồ theo ngày, p-value, kết luận có ý nghĩa thống kê hay chưa	`GET /api/ab/ket-qua`
Chất lượng mô hình	NDCG/Precision/Coverage/Novelty theo phiên bản mô hình; biểu đồ phân phối độ phổ biến của item được gợi ý	`GET /api/mo-hinh/chi-so`
Tái lập thí nghiệm	Chọn 1 kết quả A/B trong quá khứ → hiển thị phiên bản Delta của dữ liệu → nút *Chạy lại* → so sánh kết quả cũ và mới	`POST /api/tai-lap/{ngay}`

Phân nhóm A/B: hash `ma_nguoi_dung` → nhóm A (ALS thuần) hoặc nhóm B (ALS trộn thịnh hành 30%). Phân nhóm phải **ổn định** — cùng user luôn vào cùng nhóm.

3.9. Đặc tả Airflow

DAG 1 — `kn_pipeline_hang_ngay` (`0 2 * * *`)

```
cho_du_lieu_ngay_moi (Sensor)
        │
        ▼
bronze_to_silver ──► thong_ke_san_pham
        │
        ▼
chuan_bi_ma_tran ──► huan_luyen_als ──► danh_gia
                                          │
                          [BranchPythonOperator: NDCG có cải thiện?]
                           ├── có ──► sinh_goi_y_toan_bo ──► tron_goi_y ──► ghi_gold
                           └── không ──► giu_mo_hinh_cu ──► canh_bao
```

DAG 2 — `kn_giam_sat_streaming` (`*/15 * * * *`)

Kiểm tra job thịnh hành còn sống, kiểm tra `gold.thinh_hanh_15p` có dữ liệu trong 15 phút qua.

DAG 3 — `kn_tinh_ab` (`0 * * * *`)

Đọc CDF của impression log → tổng hợp → z-test → ghi `gold.ket_qua_ab` kèm `phien_ban_delta_du_lieu`.

DAG 4 — `kn_bao_tri` (`0 4 * * *`)

`OPTIMIZE` + `ZORDER` + `VACUUM` + xóa gợi ý cũ hơn 7 ngày.

DAG 5 — `kn_tai_lap` (thủ công)

Nhận `phien_ban_delta` qua `dag_run.conf`, chạy lại toàn bộ pipeline huấn luyện trên phiên bản đó, so sánh metric với bản ghi lịch sử. **Bắt buộc demo.**

3.10. Rủi ro và phương án dự phòng

Rủi ro	Phương án
ALS OOM với 1,4 triệu user	Lọc user có ≥ 3 tương tác (giảm còn ~300k); tăng `numBlocks`; ghi rõ trong báo cáo
Không đủ traffic để A/B có ý nghĩa thống kê	Viết script mô phỏng người dùng ảo click theo mô hình xác suất phụ thuộc vị trí và độ liên quan; ghi rõ đây là traffic mô phỏng
`recommendForAllUsers` quá chậm	Chỉ sinh cho user hoạt động trong 30 ngày; user còn lại lấy fallback theo danh mục
Coverage quá thấp (popularity bias nặng)	Đây là **kết quả hợp lệ** nếu được phân tích: trình bày nguyên nhân và thử ít nhất 1 biện pháp giảm bias (re-ranking theo novelty)

3.11. Danh mục nghiệm thu

- ☐ Bảng tuning ≥ 9 tổ hợp ALS với NDCG@10
- ☐ So sánh ALS vs 2 baseline theo 4 metric xếp hạng + Coverage + Novelty
- ☐ Chứng minh 3 chiến lược cold start hoạt động (demo với user mới tạo tại chỗ)
- ☐ Streaming thịnh hành cập nhật trong ≤ 90 giây sau khi bấm nút trên web
- ☐ Vòng khép kín: bấm *Thêm giỏ* trên web → sự kiện vào Kafka → xuất hiện trong bronze → ảnh hưởng bảng thịnh hành
- ☐ Kết quả A/B có p-value và kết luận thống kê
- ☐ Demo tái lập thí nghiệm bằng `VERSION AS OF` cho ra **cùng** metric
- ☐ Số liệu OPTIMIZE/ZORDER trên truy vấn lấy gợi ý theo user

ĐỀ TÀI 4 — PHÂN TÍCH VÀ DỰ BÁO NHU CẦU DI CHUYỂN ĐÔ THỊ

Mã đề tài: `bd04_giao_thong` **Độ khó:** Trung bình–Cao **Trọng tâm kỹ thuật:** Xử lý dữ liệu quy mô lớn, tối ưu hiệu năng Spark/Delta, backfill nhiều năm bằng Airflow

4.1. Bối cảnh nghiệp vụ

Một nền tảng gọi xe cần dự báo **số chuyến phát sinh trong 1–2 giờ tới cho từng vùng của thành phố** để điều phối tài xế đến nơi sắp có nhu cầu cao, giảm thời gian chờ và tỷ lệ hủy chuyến. Bài toán phụ: dự báo thời gian di chuyển (ETA) giữa hai vùng.

Đề tài này khác 3 đề tài trên ở chỗ **quy mô dữ liệu là trọng tâm**: dữ liệu gốc nhiều chục triệu đến hàng trăm triệu bản ghi, trải nhiều năm, schema thay đổi giữa các năm. Đây là đề tài phù hợp nhất để thể hiện năng lực tối ưu Spark và Delta.

4.2. Nguồn dữ liệu

Chính: *NYC TLC Trip Record Data* — dữ liệu công khai chính thức, định dạng Parquet theo tháng. `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`

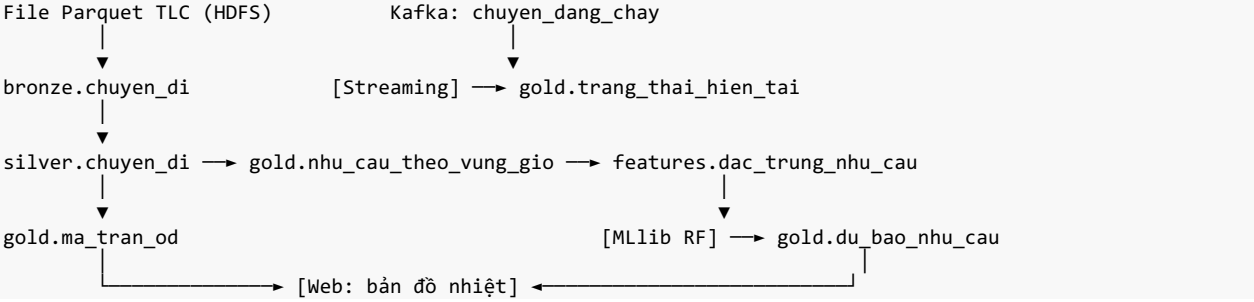
- Yellow Taxi: ~3 triệu chuyến/tháng
- **Yêu cầu tối thiểu: 24 tháng dữ liệu** (khoảng 70–100 triệu bản ghi). Nhóm nào chỉ nạp 1–2 tháng sẽ bị trừ điểm hạng mục hiệu năng.
- Kèm `taxi_zone_lookup.csv` (265 vùng) và shapefile vùng để vẽ bản đồ.

Điểm quan trọng: schema TLC **thay đổi thật** giữa các năm (đổi tên cột, thêm `airport_fee`, `congestion_surcharge`). Đây là tình huống schema evolution tự nhiên, không cần mô phỏng.

Bổ sung: dữ liệu thời tiết theo giờ của New York (Open-Meteo Historical API), lịch ngày lễ Mỹ.

Streaming: phát lại chuyến đi của tháng gần nhất qua Kafka theo thời gian thực nén (1 giờ = 20 giây) để mô phỏng chuyến đang diễn ra.

4.3. Kiến trúc hệ thống



4.4. Đặc tả bảng dữ liệu

bronze.chuyen_di

Giữ nguyên schema từng tháng, dùng `mergeSchema` khi hợp nhất. Phân vùng theo `nam`, `thang`.

silver.chuyen_di

Đã chuẩn hóa về schema thống nhất.

Cột	Kiểu	Mô tả
ma_chuyen	STRING	Sinh từ hash
thoi_diem_don, thoi_diem_tra	TIMESTAMP	
vung_don, vung_tra	INT	1–265
so_khach	INT	
quang_duong_mile	DOUBLE	
thoi_gian_phut	DOUBLE	Tính từ 2 timestamp
van_toc_tb_mph	DOUBLE	
tong_tien	DOUBLE	
hinh_thuc_tt	STRING	
nam, thang, ngay	INT/DATE	Cột phân vùng

Quy tắc làm sạch (bắt buộc ghi rõ số bản ghi bị loại ở mỗi bước): - `thoi_diem_tra <= thoi_diem_don` → loại - `thoi_gian_phut < 1` hoặc `> 300` → loại - `quang_duong_mile <= 0` hoặc `> 100` → loại - `van_toc_tb_mph > 80` → loại - `so_khach = 0` hoặc `> 8` → loại - `vung_don` hoặc `vung_tra` không thuộc 1–263 (loại bỏ vùng "Unknown" 264/265) → `_quarantine` - `tong_tien < 0` → loại (chuyến hoàn tiền)

gold.nhu_cau_theo_vung_gio

Bảng sự thật trung tâm. Lưới đầy đủ: mọi (vùng × giờ), kể cả giờ không có chuyến (điền 0) — nếu không, mô hình sẽ học lệch.

Cột	Kiểu
vung	INT
khung_gio	TIMESTAMP (làm tròn giờ)
so_chuyen	INT
tong_doanh_thu	DOUBLE
thoi_gian_chuyen_tb	DOUBLE
quang_duong_tb	DOUBLE

gold.ma_tran_od (Origin–Destination)

Cột	Kiểu
vung_don, vung_tra	INT
khung_gio_trong_ngay	INT (0–23)
thu_trong_tuan	INT
so_chuyen	LONG
thoi_gian_trung_vi	DOUBLE

Cột	Kiểu
thoi_gian_p90	DOUBLE

features.dac_trung_nhu_cau

Nhóm	Chi tiết
Trễ	lag_1h, lag_2h, lag_24h, lag_168h (cùng giờ tuần trước)
Trượt	trung bình/độ lệch chuẩn 3h, 24h, 168h
Không gian	tổng nhu cầu của các vùng **kề** (từ ma trận kề dựng từ shapefile) ở giờ trước — đặc trưng lan truyền không gian
Thời gian	giờ, thứ, tháng, la_cuoi_tuan, la_ngay_le, la_gio_cao_diem
Chu kỳ	sin/cos của giờ và thứ
Thời tiết	nhiệt độ, lượng mưa, tốc độ gió tại khung_gio
Vùng	loại vùng (sân bay / trung tâm / dân cư — từ bảng lookup)

gold.du_bao_nhu_cau

Cột	Kiểu
thoi_diem_du_bao	TIMESTAMP
vung	INT
khung_gio_muc_tieu	TIMESTAMP
horizon_gio	INT (1 hoặc 2)
so_chuyen_du_bao	DOUBLE
can_duoi, can_tren	DOUBLE
phien_ban_mo_hinh	STRING

4.5. Đặc tả các job Spark và yêu cầu hiệu năng

Job	Loại	Yêu cầu
ingest/nap_thang.py	Batch	Nạp 1 tháng Parquet → bronze; idempotent (chạy lại không nhân đôi dữ liệu)
transform/bronze_to_silver.py	Batch	Chuẩn hóa schema đa năm, làm sạch, ghi thống kê loại bỏ
transform/tao_nhu_cau.py	Batch	Sinh lưới đầy đủ vùng × giờ (cross join với bảng lịch)
transform/tao_ma_tran_od.py	Batch	Tổng hợp OD
ml/tao_dac_trung.py	Batch	Window functions + join ma trận kề + thời tiết
ml/huan_luyen.py	Batch	RandomForestRegressor + baseline
ml/du_bao.py	Batch	Dự báo cho toàn bộ 263 vùng
streaming/theo_doi_chuyen.py	Streaming	Đếm chuyến đang chạy theo vùng, cửa sổ 5 phút

Hạng mục hiệu năng bắt buộc (chấm điểm riêng — đây là điểm khác biệt của đề tài này)

Nhóm phải thực hiện và **báo cáo bằng số liệu** ít nhất 5 trong 7 thí nghiệm sau:

1. **Partition pruning**: đo thời gian truy vấn 1 tháng khi có/không phân vùng theo `nam, thang`. Chụp `EXPLAIN` cho thấy số partition được quét.
2. **Small file problem**: đo số file và kích thước trung bình trước/sau `OPTIMIZE`; đo lại thời gian truy vấn.
3. **Z-ORDER**: `ZORDER BY (vung_don, thoi_diem_don)`; đo truy vấn "tất cả chuyến từ vùng 132 trong tháng 3".
4. **Broadcast join**: join với `taxi_zone_lookup` (265 dòng) — so sánh có/không `broadcast()`, xem `EXPLAIN` để xác nhận `BroadcastHashJoin`.
5. **Data skew**: vùng sân bay JFK (132) và Manhattan Midtown có lượng chuyến lớn bất thường. Đo thời gian task theo partition, phát hiện skew, xử lý bằng salting hoặc AQE (`spark.sql.adaptive.skewJoin.enabled`). So sánh trước/sau.
6. **Cache/persist**: đo lợi ích khi cache bảng đặc trưng dùng lại nhiều lần trong tuning.
7. **File format**: so sánh thời gian đọc cùng dữ liệu ở dạng CSV vs Parquet vs Delta.

Mỗi thí nghiệm cần: mô tả cách đo, số liệu trước/sau, ảnh chụp Spark UI, và **giải thích tại sao**.

4.6. Đặc tả mô hình học máy

Mô hình A — Dự báo nhu cầu theo vùng

- **Thuật toán:** `RandomForestRegressor`. So sánh với `GBTRegressor` và 2 baseline: (1) giá trị cùng giờ tuần trước, (2) trung bình cùng (vùng, giờ, thứ) trong 4 tuần gần nhất.
- **Biến mục tiêu:** `so_chuyen`. Xem xét biến đổi `log1p` vì phân phối lệch phải mạnh — nhóm phải thử và báo cáo có/không.
- **Chia dữ liệu:** theo thời gian. 18 tháng train, 3 tháng validation, 3 tháng test.
- **Metric:** MAE và RMSE toàn cục; **quan trọng hơn**: báo cáo riêng theo 3 nhóm vùng (nhu cầu cao / trung bình / thấp), vì sai số ở vùng nhu cầu thấp bị metric toàn cục che khuất.
- **Ngưỡng đạt:** MAE tốt hơn baseline tuần trước ≥ 25% ở nhóm vùng nhu cầu cao, và ≥ 10% ở nhóm nhu cầu thấp.
- **Bắt buộc phân tích:** biểu đồ residual theo giờ trong ngày và theo vùng, chỉ ra thời điểm/khu vực mô hình yếu nhất và giải thích.

Mô hình B — Dự báo thời gian chuyến (ETA)

- `GBTRegressor` với đầu vào: vùng đón, vùng trả, giờ, thứ, quãng đường, thời tiết.
- Metric: MAE (phút), MAPE. Ngưỡng đạt: MAE ≤ 4 phút trên tập test.
- So sánh với baseline: thời gian trung vị lịch sử của cặp OD trong cùng khung giờ (`gold.ma_tran_od`).

4.7. Đặc tả Delta Lake

Yêu cầu	Cách thực hiện
Schema evolution (tự nhiên)	Schema TLC khác nhau giữa các năm. Nạp bằng `mergeSchema=true`, ghi lại `DESCRIBE HISTORY` cho thấy các lần thay đổi schema, lập bảng đối chiếu tên cột giữa các năm
MERGE	Idempotent ingest: `MERGE` theo `ma_chuyen` để chạy lại tháng không nhân đôi bản ghi. Phải demo: chạy DAG nạp cùng 1 tháng 2 lần, số bản ghi không đổi

Yêu cầu	Cách thực hiện
Time travel	Trước/sau khi backfill dữ liệu sửa đổi của TLC — so sánh dự báo sinh từ 2 phiên bản dữ liệu
OPTIMIZE / Z-ORDER	Xem mục 4.5, thí nghiệm 2 và 3
Partition	`nam`/`thang` trên bronze và silver; giải thích vì sao **không** phân vùng theo `ngay` (quá nhiều partition với 24 tháng)
VACUUM	Sau khi hoàn tất backfill, đo dung lượng HDFS trước/sau VACUUM
Generated columns	`nam`, `thang` là generated column từ `thoi_diem_don` — chứng minh partition pruning vẫn hoạt động khi truy vấn theo `thoi_diem_don`

4.8. Đặc tả ứng dụng web

Màn hình	Nội dung	API
Bản đồ nhiệt nhu cầu	Bản đồ 263 vùng NYC (Leaflet + GeoJSON), tô màu theo nhu cầu; thanh trượt chọn khung giờ; nút chuyển giữa *Thực tế* / *Dự báo* / *Sai số*	`GET /api/nhu-cau?khung_gio=&che_do=`
Chi tiết vùng	Chọn 1 vùng → biểu đồ nhu cầu 7 ngày qua + dự báo 2 giờ tới, top 5 điểm đến phổ biến từ vùng đó, thời gian chuyến trung bình	`GET /api/vung/{id}`
Ma trận OD	Heatmap 263×263 (hoặc rút gọn top 30 vùng), lọc theo khung giờ và thứ; nhấn ô để xem phân phối thời gian chuyến	`GET /api/od?gio=&thu=`
Điều phối (ghi ngược)	Người điều phối chọn vùng và số xe muốn điều đến, ghi **`silver.ke_hoach_dieu_phoi`**; màn hình hôm sau đối chiếu kế hoạch vs nhu cầu thực tế	`POST /api/dieu-phoi`
Hiệu năng hệ thống	Kết quả 7 thí nghiệm hiệu năng ở mục 4.5, dung lượng HDFS theo tầng, lịch sử chạy DAG	`GET /api/he-thong/thong-ke`

4.9. Đặc tả Airflow

DAG 1 — `gt_nap_thang_moi` (`0 6 5 * *` — ngày 5 hằng tháng)

```
FileSensor: chờ file Parquet tháng mới trên HDFS
        │
        ▼
kiem_tra_schema (so với schema chuẩn)
        │
[BranchPythonOperator]
        ├── schema khớp ──► nap_bronze
        └── schema khác ──► ghi_nhat_ky_thay_doi ──► nap_bronze_mergeSchema
        │
        ▼
bronze_to_silver ──► tao_nhu_cau ──► tao_ma_tran_od ──► tao_dac_trung
```

DAG 2 — `gt_backfill` (thủ công, bắt buộc demo)

Nhận `nam_bat_dau`, `thang_bat_dau`, `nam_ket_thuc`, `thang_ket_thuc`. Dùng **dynamic task mapping** (`.expand()` của Airflow 2.x) để sinh song song 1 task cho mỗi tháng, giới hạn `max_active_tis_per_dag = 2` tránh quá tải cụm. Phải demo backfill ≥ 6 tháng.

DAG 3 — `gt_du_bao` (`0,30 * * * *`)

Sinh dự báo horizon 1h và 2h cho toàn bộ 263 vùng, ghi gold, cập nhật cache web. `sla = 15 phút`.

DAG 4 — `gt_huan_luyen_lai` (`0 3 1 * *` — đầu tháng)

Huấn luyện lại trên dữ liệu 24 tháng gần nhất, so sánh MAE với mô hình hiện tại, promote nếu tốt hơn.

DAG 5 — `gt_bao_tri` (`0 5 * * 0`)

`OPTIMIZE` + `ZORDER` toàn bộ bảng lớn, `VACUUM`, ghi thống kê dung lượng.

4.10. Rủi ro và phương án dự phòng

Rủi ro	Phương án
Không đủ dung lượng đĩa cho 24 tháng	Giảm còn 12 tháng nhưng **phải giữ ít nhất 2 năm khác nhau** để có schema evolution; hoặc lấy mẫu 30% mỗi tháng (ghi rõ cách lấy mẫu)
Job chạy quá lâu (> 2 giờ)	Đây là cơ hội, không phải rủi ro — chính là nội dung của mục 4.5. Tối ưu và báo cáo
Shapefile / bản đồ khó dựng	Dùng file GeoJSON đã chuyển sẵn của NYC Taxi Zones (có sẵn công khai); nếu vẫn khó, thay bản đồ bằng heatmap dạng lưới, trừ 5% điểm web
Skew ở vùng sân bay làm job treo	Bật AQE (`spark.sql.adaptive.enabled=true`) và báo cáo mức cải thiện

4.11. Danh mục nghiệm thu

- ☐ `hdfs dfs -du -h` cho thấy ≥ 15 GB dữ liệu trong lakehouse
- ☐ Bảng đối chiếu schema TLC giữa các năm + `DESCRIBE HISTORY` có lần schema evolution
- ☐ Demo idempotent: chạy lại DAG nạp cùng 1 tháng, `COUNT(*)` không đổi
- ☐ Báo cáo ≥ 5/7 thí nghiệm hiệu năng, mỗi thí nghiệm có số liệu + Spark UI + giải thích
- ☐ Bảng thống kê làm sạch: số bản ghi bị loại ở từng quy tắc
- ☐ MAE dự báo theo 3 nhóm vùng, so với 2 baseline
- ☐ Biểu đồ residual theo giờ và theo vùng + phân tích
- ☐ MAE ETA so với baseline trung vị OD
- ☐ Demo backfill ≥ 6 tháng với dynamic task mapping
- ☐ Bản đồ nhiệt phản hồi < 3 giây khi đổi khung giờ

ĐỀ TÀI 5 — THEO DÕI XU HƯỚNG VÀ CẢM XÚC TIN TỨC TIẾNG VIỆT

Mã đề tài: `bd05_tin_tuc` **Độ khó:** Trung bình **Trọng tâm kỹ thuật:** Xử lý ngôn ngữ tự nhiên tiếng Việt trên Spark, khử trùng lặp nội dung, phát hiện đột biến chủ đề

5.1. Bối cảnh nghiệp vụ

Một bộ phận truyền thông cần công cụ theo dõi:

- **Xu hướng chủ đề**: chủ đề nào đang nổi lên trên báo điện tử Việt Nam trong 24 giờ qua.
- **Cảm xúc theo từ khóa**: với một thương hiệu / nhân vật / chủ đề do người dùng nhập, tỷ lệ tin tích cực – trung tính – tiêu cực biến động ra sao.
- **Cảnh báo đột biến**: khi một chủ đề tăng bất thường (khủng hoảng truyền thông), gửi cảnh báo trong vòng 15 phút.

Thách thức đặc thù: tiếng Việt có **tách từ ghép** (không tách theo khoảng trắng), báo điện tử **đăng lại chéo nhau** (cùng một tin xuất hiện ở 10 báo với tiêu đề khác nhau), và **nhãn cảm xúc khan hiếm**.

5.2. Nguồn dữ liệu

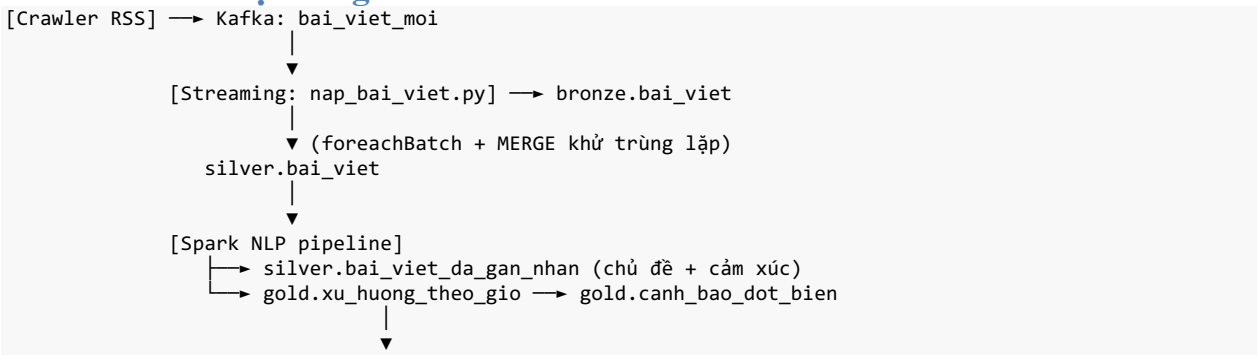
Thu thập trực tiếp: RSS của các báo điện tử Việt Nam (VnExpress, Tuổi Trẻ, Thanh Niên, VietnamNet, Dân Trí, Zing…). Mỗi báo có endpoint RSS công khai theo chuyên mục. Crawler chạy 15 phút/lần.

Có nhãn (cho huấn luyện): - *VLSP Sentiment Analysis* (bình luận sản phẩm, 3 lớp) — `https://vlsp.org.vn/resources` - *UIT-VSFC* — 16.000 câu phản hồi sinh viên có nhãn cảm xúc, tiếng Việt - *VNTC* (VN News Corpus) — ~33.000 bài báo tiếng Việt có nhãn 10 chuyên mục, dùng cho phân loại chủ đề

Lưu ý pháp lý và đạo đức (bắt buộc trong báo cáo): chỉ thu thập nội dung công khai qua RSS; tôn trọng `robots.txt`; giới hạn tần suất request; **không lưu thông tin cá nhân của người bình luận**; không thu thập nội dung sau tường phí. Nhóm phải có mục *Đạo đức dữ liệu* trong báo cáo.

Bootstrap: vì crawl 8 tuần chỉ được ~50.000 bài, nhóm nạp thêm bộ VNTC vào bronze để có đủ khối lượng cho huấn luyện.

5.3. Kiến trúc hệ thống



```
                  [Web: dashboard + theo dõi từ khóa]
                                  |
                                  ▼
                  silver.tu_khoa_theo_doi + silver.phan_hoi_nhan
```

5.4. Đặc tả bảng dữ liệu

bronze.bai_viet

Cột	Kiểu
ma_bai	STRING (UUID)
url	STRING
nguon	STRING (tên báo)
chuyen_muc_goc	STRING
tieu_de	STRING
mo_ta	STRING
noi_dung	STRING
tac_gia	STRING
thoi_diem_dang	TIMESTAMP
thoi_diem_thu_thap	TIMESTAMP
ngay	DATE (phân vùng)

silver.bai_viet

Đã khử trùng lặp và chuẩn hóa.

Cột bổ sung	Kiểu	Mô tả
hash_noi_dung	STRING	SHA-256 của nội dung đã chuẩn hóa (bỏ dấu câu, lower, bỏ khoảng trắng thừa)
simhash	LONG	SimHash 64-bit để phát hiện gần trùng
ma_bai_goc	STRING	Nếu là bản đăng lại, trỏ về bài xuất hiện sớm nhất
la_ban_goc	BOOLEAN	
so_tu	INT	
noi_dung_sach	STRING	Đã bỏ HTML, quảng cáo, chú thích ảnh

Khử trùng lặp 2 tầng (chấm điểm riêng): 1. **Trùng chính xác**: `hash_noi_dung` giống nhau → gộp 2. **Gần trùng**: khoảng cách Hamming của SimHash ≤ 3 **hoặc** Jaccard similarity của tập 3-gram ≥ 0,8 → gộp, giữ bài có `thoi_diem_dang` sớm nhất làm bản gốc

Nhóm phải báo cáo: tổng số bài thu thập, số bài sau khử trùng lặp, tỷ lệ đăng lại chéo giữa các báo.

silver.bai_viet_da_gan_nhan

Cột	Kiểu

Cột	Kiểu
ma_bai	STRING
chu_de_du_doan	STRING
do_tin_cay_chu_de	DOUBLE
cam_xuc	STRING (tich_cuc/trung_tinh/tieu_cuc)
diem_cam_xuc	DOUBLE (−1..1)
do_tin_cay_cam_xuc	DOUBLE
chu_de_lda	INT
tu_khoa_chinh	ARRAY
phien_ban_mo_hinh	STRING

gold.xu_huong_theo_gio

Cột	Kiểu
khung_gio	TIMESTAMP
chu_de	STRING
so_bai	INT
so_nguon_khac_nhau	INT
diem_cam_xuc_tb	DOUBLE
ty_le_tieu_cuc	DOUBLE
tu_khoa_noi_bat	ARRAY
diem_dot_bien	DOUBLE

`diem_dot_bien = (so_bai_gio_nay − trung_binh_168h) / (do_lech_chuan_168h + 1)` — z-score so với 7 ngày cùng khung giờ.

gold.canh_bao_dot_bien

Cột	Kiểu
ma_canh_bao	STRING
khung_gio	TIMESTAMP
chu_de	STRING
tu_khoa	ARRAY
diem_dot_bien	DOUBLE
ty_le_tieu_cuc	DOUBLE
bai_tieu_bieu	ARRAY (URL)
trang_thai	STRING

silver.tu_khoa_theo_doi (ghi từ web)

Cột	Kiểu
ma_tu_khoa	STRING

Cột	Kiểu
tu_khoa	STRING
nguoi_tao	STRING
nguong_canh_bao	DOUBLE
dang_hoat_dong	BOOLEAN
tao_luc	TIMESTAMP

silver.phan_hoi_nhan (ghi từ web — vòng phản hồi)

Cột	Kiểu
ma_bai	STRING
cam_xuc_du_doan	STRING
cam_xuc_dung	STRING
nguoi_gan_nhan	STRING
thoi_diem	TIMESTAMP

5.5. Đặc tả các job Spark

Job	Loại	Yêu cầu
ingest/crawler.py	Python thuần	Đọc RSS, lấy nội dung đầy đủ, đẩy Kafka; tôn trọng rate limit
streaming/nap_bai_viet.py	Streaming	Kafka → bronze; foreachBatch gọi MERGE khử trùng lặp vào silver
transform/khu_trung_lap.py	Batch	SimHash + Jaccard; job này chạy lại toàn bộ 48h gần nhất mỗi đêm để bắt các bản đăng lại muộn
ml/tien_xu_ly.py	Batch	Tách từ tiếng Việt (pandas UDF gọi underthesea/pyvi), bỏ stopword, chuẩn hóa dấu
ml/huan_luyen_chu_de.py	Batch	Phân loại chuyên mục
ml/huan_luyen_cam_xuc.py	Batch	Phân loại cảm xúc
ml/lda_chu_de.py	Batch	LDA của MLlib phát hiện chủ đề nổi không cần nhãn
ml/gan_nhan.py	Batch	Áp mô hình lên bài mới
transform/tinh_xu_huong.py	Batch	Tổng hợp gold + tính điểm đột biến

Điểm kỹ thuật quan trọng — tách từ tiếng Việt trên Spark: underthesea là thư viện Python thuần, không phân tán được trực tiếp. Nhóm phải dùng **Pandas UDF (applyInPandas hoặc pandas_udf)** để xử lý theo lô, và **đo lường** chênh lệch hiệu năng so với UDF thường (row-at-a-time). Đây là hạng mục chấm điểm riêng.

5.6. Đặc tả mô hình học máy

Mô hình A — Phân loại chuyên mục

- **Đặc trưng:** Tokenizer (sau khi tách từ tiếng Việt) → StopWordsRemover (danh sách stopword tiếng Việt) → HashingTF/CountVectorizer → IDF. Bổ sung: thử Word2Vec của MLlib và so sánh.

- **Thuật toán:** `LogisticRegression` (multinomial) và `NaiveBayes`. So sánh cả hai.
- **Dữ liệu:** VNTC (10 chuyên mục), chia 70/15/15.
- **Metric:** Accuracy, Macro-F1, ma trận nhầm lẫn.
- **Ngưỡng đạt:** Macro-F1 ≥ 0,80 trên tập test.

Mô hình B — Phân loại cảm xúc

- **Dữ liệu huấn luyện:** UIT-VSFC / VLSP (3 lớp).
- **Thuật toán:** `LogisticRegression` trên TF-IDF làm baseline; so sánh với TF-IDF + `LinearSVC`.
- **Metric:** Macro-F1 (vì 3 lớp mất cân bằng), báo cáo F1 từng lớp.
- **Ngưỡng đạt:** Macro-F1 ≥ 0,70 trên tập test của bộ có nhãn.
- **Vấn đề bắt buộc phân tích — domain shift:** mô hình huấn luyện trên bình luận/phản hồi nhưng áp dụng lên **tin tức**. Nhóm phải: (1) gán nhãn thủ công 200 bài báo ngẫu nhiên, (2) đo Macro-F1 trên tập này, (3) báo cáo mức sụt giảm và thảo luận nguyên nhân. Sụt giảm là kết quả **được chấp nhận** nếu phân tích tốt.
- **Cải thiện bằng vòng phản hồi:** nhãn từ `silver.phan_hoi_nhan` được đưa vào tập huấn luyện; báo cáo Macro-F1 theo tuần.

Mô hình C — LDA phát hiện chủ đề

- `LDA` của MLlib, `k` chọn trong {10, 15, 20, 25} theo perplexity trên tập held-out.
- Với mỗi chủ đề, trích 10 từ đại diện; nhóm **đặt tên thủ công** cho từng chủ đề và trình bày bảng.
- Đánh giá định tính: chọn 5 chủ đề, kiểm tra 10 bài có xác suất cao nhất có thực sự cùng chủ đề không.

5.7. Đặc tả Delta Lake

Yêu cầu	Cách thực hiện
MERGE	`silver.bai_viet`: `WHEN NOT MATCHED THEN INSERT` theo `hash_noi_dung`; khi phát hiện bản đăng lại muộn → `UPDATE` trường `ma_bai_goc`, `la_ban_goc`
Time travel	Tái dựng **bảng xếp hạng chủ đề nổi tại một thời điểm quá khứ**: dữ liệu tiếp tục về sau đó sẽ làm thay đổi kết quả nếu tính lại. Web có màn hình "Xem lại bảng xu hướng lúc HH:MM ngày DD/MM" dùng `VERSION AS OF`
Schema evolution	Tuần 5 thêm `so_luot_binh_luan`, `so_luot_chia_se`; mô hình v2 dùng thêm tín hiệu tương tác để tính điểm đột biến
Change Data Feed	Job `tinh_xu_huong.py` chỉ đọc bài mới/bài được cập nhật thay vì quét lại 7 ngày
OPTIMIZE / Z-ORDER	`ZORDER BY (chu_de, thoi_diem_dang)`; đo truy vấn "tất cả bài về chủ đề X trong 30 ngày"
Xóa theo yêu cầu	Báo gỡ bài → `DELETE FROM silver.bai_viet WHERE url = ...` + `VACUUM`; chứng minh dữ liệu thực sự bị xóa khỏi HDFS. Đây là yêu cầu tuân thủ thực tế

5.8. Đặc tả ứng dụng web

Màn hình	Nội dung	API
Bảng xu	Top 10 chủ đề 24h qua, biểu đồ số bài theo giờ, chỉ báo	`GET /api/xu-huong?tu=&den=`

Màn hình	Nội dung	API
hướng	đột biến (mũi tên + z-score), word cloud từ khóa nổi bật	
Theo dõi từ khóa	Người dùng nhập từ khóa (VD: tên thương hiệu) → biểu đồ số bài và tỷ lệ cảm xúc theo ngày, danh sách bài liên quan; **lưu từ khóa vào** `silver.tu_khoa_theo_doi` với ngưỡng cảnh báo tùy chỉnh	`POST /api/tu-khoa`, `GET /api/tu-khoa/{id}/thong-ke`
Cảnh báo đột biến	Danh sách cảnh báo, mỗi cảnh báo có chủ đề, từ khóa, 5 bài tiêu biểu, tỷ lệ tiêu cực; nút *Đã xem* / *Bỏ qua*	`GET /api/canh-bao`, `POST /api/canh-bao/{id}/trang-thai`
Gán nhãn (ghi ngược)	Hiển thị bài cùng nhãn cảm xúc dự đoán, người dùng sửa nhãn nếu sai → **ghi** `silver.phan_hoi_nhan`	`POST /api/bai/{id}/nhan`
Xem lại lịch sử (time travel)	Chọn ngày giờ → hiển thị bảng xu hướng **đúng như hệ thống thấy tại thời điểm đó**, so sánh với bảng tính lại từ dữ liệu đầy đủ hiện nay	`GET /api/lich-su?thoi_diem=`
Chất lượng mô hình	Macro-F1 theo tuần, ma trận nhầm lẫn, số nhãn phản hồi đã thu thập	`GET /api/mo-hinh/chi-so`

5.9. Đặc tả Airflow

DAG 1 — `tt_crawl` (`*/15 * * * *`)

```
crawl_rss (TaskGroup, 1 task/báo, chạy song song)
        │
        ▼
kiem_tra_so_luong (số bài mới > 0?)
        │
        ▼
day_kafka
```

Mỗi task báo có `retries=3`, `execution_timeout=5 phút`. Một báo lỗi **không được** làm hỏng cả DAG → dùng `trigger_rule='all_done'` ở task sau.

DAG 2 — `tt_gan_nhan` (`*/30 * * * *`)

```
khu_trung_lap ──► tien_xu_ly ──► gan_nhan_chu_de ──► gan_nhan_cam_xuc ──► tinh_xu_huong
                                                                                │
                                                                  [Branch: có đột biến?]
                                                                    ├── có ──► ghi_canh_bao ──►
gui_email
                                                                    └── không ──► ket_thuc
```

`sla = 20 phút` (yêu cầu cảnh báo trong 15 phút → toàn bộ chuỗi phải xong nhanh).

DAG 3 — `tt_khu_trung_lap_sau` (`0 2 * * *`)

Quét lại 48 giờ gần nhất để bắt bản đăng lại muộn, cập nhật `ma_bai_goc` bằng MERGE, tính lại `gold.xu_huong_theo_gio` cho các giờ bị ảnh hưởng.

DAG 4 — `tt_huan_luyen_lai` (`0 3 * * 0`)

Kiểm tra có ≥ 200 nhãn phản hồi mới → huấn luyện lại mô hình cảm xúc → so sánh Macro-F1 → promote hoặc giữ nguyên.

DAG 5 — `tt_lda` (`0 4 * * 0`)

Chạy lại LDA trên toàn bộ corpus tuần qua, cập nhật danh sách chủ đề, cảnh báo nếu xuất hiện chủ đề mới không khớp chủ đề cũ.

DAG 6 — `tt_bao_tri` (`0 5 * * *`)

`OPTIMIZE` + `ZORDER` + `VACUUM` + xóa bài cũ hơn 180 ngày.

5.10. Rủi ro và phương án dự phòng

Rủi ro	Phương án
Báo chặn crawler	Giảm tần suất, đổi User-Agent hợp lệ có thông tin liên hệ, chỉ dùng RSS; nếu vẫn bị chặn → giảm số nguồn, ghi rõ trong báo cáo
Không đủ dữ liệu crawl trong 8 tuần	Nạp VNTC (33.000 bài) vào bronze làm dữ liệu lịch sử; ghi rõ phần nào là crawl thật, phần nào là bộ dữ liệu có sẵn
`underthesea` chậm, không phân tán tốt	Bắt buộc dùng Pandas UDF; nếu vẫn chậm, giới hạn ở tiêu đề + mô tả thay vì toàn văn (trừ 5% điểm)
Macro-F1 cảm xúc thấp do domain shift	Kết quả hợp lệ nếu có phân tích đầy đủ theo mục 5.6
Không có đột biến nào xảy ra trong kỳ demo	Chuẩn bị sẵn dữ liệu lịch sử của một sự kiện đã xảy ra và phát lại để demo cảnh báo

5.11. Danh mục nghiệm thu

- ☐ Thống kê crawl: số bài, số nguồn, tỷ lệ trùng lặp chính xác, tỷ lệ gần trùng
- ☐ So sánh hiệu năng Pandas UDF vs UDF thường khi tách từ (số liệu cụ thể)
- ☐ Macro-F1 phân loại chuyên mục + ma trận nhầm lẫn
- ☐ Macro-F1 cảm xúc trên tập gốc **và** trên 200 bài báo gán nhãn thủ công + phân tích domain shift
- ☐ Bảng 10 chủ đề LDA với từ đại diện và tên do nhóm đặt
- ☐ Demo cảnh báo đột biến: phát lại sự kiện → cảnh báo xuất hiện trong ≤ 15 phút
- ☐ Demo time travel: bảng xu hướng lúc quá khứ khác bảng tính lại hiện tại, giải thích được vì sao
- ☐ Demo xóa bài theo yêu cầu + `VACUUM` + chứng minh dữ liệu không còn trên HDFS
- ☐ Mục *Đạo đức dữ liệu* trong báo cáo
- ☐ Demo: sửa nhãn trên web → nhãn vào Delta → DAG huấn luyện lại dùng được

PHẦN C. PHỤ LỤC

C.1. Bảng so sánh 5 đề tài

Tiêu chí	ĐT1 Điện lực	ĐT2 Gian lận	ĐT3 Khuyến nghị	ĐT4 Giao thông	ĐT5 Tin tức
Độ khó	Trung bình	Cao	Trung bình	TB–Cao	Trung bình
Streaming	Bắt buộc	Bắt buộc, có trạng thái	Bắt buộc	Nhẹ	Bắt buộc
Loại bài toán ML	Hồi quy + bất thường	Phân loại mất cân bằng	Xếp hạng	Hồi quy	Phân loại văn bản + LDA
Quy mô dữ liệu	Trung bình	Trung bình	Trung bình	**Lớn**	Nhỏ–TB
Trọng tâm Delta	MERGE dữ liệu trễ	Kiểm toán/tái lập	A/B time travel	Schema evolution, tối ưu	Khử trùng lặp, xóa dữ liệu
Trọng tâm Airflow	Backfill	Branch theo drift	Tái lập thí nghiệm	Dynamic task mapping	Nhiều nguồn, chịu lỗi từng phần
Yêu cầu hạ tầng	Trung bình	Cao (Kafka + state)	Trung bình	**Cao (đĩa)**	Thấp
Phù hợp nhóm	Cân bằng	Mạnh về hệ thống	Mạnh về ML	Mạnh về data engineering	Mạnh về NLP

C.2. Cấu hình Spark tham khảo cho môi trường hạn chế

```
spark.master                            yarn
spark.submit.deployMode                 client
spark.executor.instances                2
spark.executor.memory                   2g
spark.executor.cores                    2
spark.driver.memory                     2g
spark.sql.shuffle.partitions            16
spark.sql.adaptive.enabled              true
spark.sql.adaptive.skewJoin.enabled     true
spark.serializer                        org.apache.spark.serializer.KryoSerializer
spark.sql.extensions                    io.delta.sql.DeltaSparkSessionExtension
spark.sql.catalog.spark_catalog         org.apache.spark.sql.delta.catalog.DeltaCatalog
spark.databricks.delta.retentionDurationCheck.enabled  false
```

Lệnh submit mẫu:

```
spark-submit \
  --packages io.delta:delta-spark_2.12:3.1.0 \
  --conf spark.sql.extensions=io.delta.sql.DeltaSparkSessionExtension \
  --conf spark.sql.catalog.spark_catalog=org.apache.spark.sql.delta.catalog.DeltaCatalog \
  spark_jobs/transform/bronze_to_silver.py --ngay 2026-03-15
```

C.3. Cấu trúc báo cáo cuối kỳ (15–20 trang)

1. Giới thiệu bài toán và mục tiêu (1 trang)
2. Kiến trúc hệ thống — sơ đồ và giải thích lựa chọn (2 trang)
3. Dữ liệu — nguồn, khảo sát, chất lượng, thống kê làm sạch (2–3 trang)

4. Thiết kế lakehouse — schema bronze/silver/gold, chiến lược phân vùng (2 trang)
5. Xử lý Spark — các job, điểm kỹ thuật khó, tối ưu hiệu năng có số liệu (3 trang)
6. Mô hình học máy — đặc trưng, thuật toán, tuning, kết quả so với baseline, phân tích lỗi (3–4 trang)
7. Điều phối Airflow — sơ đồ DAG, xử lý lỗi, backfill (1–2 trang)
8. Ứng dụng web — màn hình, API, hiệu năng (1–2 trang)
9. Đánh giá tổng thể, hạn chế, hướng phát triển (1 trang)
10. Phân công công việc và đóng góp từng thành viên (0,5 trang)
11. Phụ lục: `AI_USAGE.md`, hướng dẫn dựng lại hệ thống

C.4. Kịch bản demo 15 phút

Phút	Nội dung
0–2	Giới thiệu bài toán, chỉ vào sơ đồ kiến trúc
2–4	Mở Airflow UI: hiển thị các DAG, chạy 1 DAG trực tiếp
4–6	Mở terminal: `hdfs dfs -ls -R`, `DESCRIBE HISTORY`, truy vấn `VERSION AS OF`
6–8	Streaming: bật producer, cho thấy dữ liệu chảy vào bronze theo thời gian thực
8–11	Web app: đi qua các màn hình, thực hiện **thao tác ghi ngược**, quay lại Delta xác nhận bản ghi mới
11–13	Trình bày kết quả mô hình: bảng so sánh với baseline, biểu đồ phân tích lỗi
13–15	Trả lời câu hỏi phản biện

C.5. Câu hỏi phản biện mẫu

Giảng viên sẽ hỏi ít nhất 3 câu trong nhóm sau:

1. Vì sao chọn phân vùng theo cột này mà không phải cột khác? Điều gì xảy ra nếu chọn sai?
2. Nếu một bản ghi đến trễ 3 ngày, hệ thống của em xử lý thế nào? Bảng gold có tự cập nhật không?
3. Chỉ ra chính xác chỗ nào trong code có thể gây data leakage và em đã ngăn ra sao?
4. Nếu DAG thất bại ở task thứ 3/5, chạy lại DAG có gây nhân đôi dữ liệu không? Chứng minh.
5. `OPTIMIZE` cải thiện được bao nhiêu? Vì sao lại cải thiện được như vậy?
6. Mô hình của em kém baseline ở trường hợp nào? Vì sao?
7. Nếu dữ liệu tăng gấp 100 lần, chỗ nào trong hệ thống sẽ vỡ trước tiên?
8. Chỉ vào một dòng code bất kỳ trong repo và yêu cầu giải thích.